

NIPT 时点选择与女胎异常判定研究

摘 要

NIPT 是通过采集母体血液检测胎儿游离 DNA 片段以分析其染色体是否异常的产前技术，其准确性与胎儿性染色体浓度相关且受孕妇 BMI 等个体差异及检测时点影响。研究 NIPT 的时点选择与胎儿异常判定，确定不同 BMI 及多因素影响下孕妇的最佳检测时点，可提升孕妇准确性，减少治疗窗口期缩短风险，保障母婴健康。

针对问题一，首先利用 3σ 原则剔除孕周和BMI的离群值，并对Y染色体浓度进行逻辑转换，以消除比例变量的不均匀干扰。然后通过数据分析发现季节性对Y染色体浓度的影响，将检测日期划分为四季指标，并以春季为基准构造虚拟变量。进一步，采用皮尔逊相关系数分析孕妇各项指标间的关系，粗略选择模型变量。基于主成分分析对测序质量指标（GC含量、原始读段数等）进行降维处理得到最终模型变量。并通过样条回归量化孕周与浓度的非线性关系。最终，构建广义混合效应模型（LMM），得到孕周（正向影响）、BMI（负向影响）、季节（秋冬季较春季负向）、PC1（测序深度、负向）指标与Y染色体浓度的显著关系。

针对问题二，首先依据每个孕妇首次达到Y染色体浓度阈值的孕周，构建BMI和检测孕周的二元变量。然后，使用分位数分组法，根据BMI大小对样本进行粗略分组，并以组内80%分位点作为临时推荐检测时点。通过判断组间孕周差值大小，进行组间合并，最终确定四个BMI组别：[20.7, 30.9)，[30.9, 31.6)，[31.6, 35.9)，[35.9, 45.0]。在风险评估中，建立基于治疗延误、检测次数和首检未达标代价的风险函数，最小化各组内的风险函数，最终得到推荐检测时点为13w+6、13w+6、13w+6、16w+3。由于模型数据信息的不足以及个体差异所引入的随机误差，采用顺延补偿法和概率命中法进行模拟，以对风险预测函数和最佳检测时点进行修正。通过比较风险曲线，进一步分析了误差对模型的影响。结果表明，误差的引入会提高整体风险水平，但对低BMI组的影响相对较小。

针对问题三，在问题二的基础上，引入了身高、体重等多种影响因素，进一步细化BMI分组并优化NIPT的最佳检测时点。考虑到不同指标间的线性与非线性关系，构建了基于分位数回归树集成方法的NIPT最佳时点优化模型。通过最小化分位数损失函数，我们将样本分为六个组别：[20.7, 30.0)、[30.0, 30.5)、[30.5, 31.2)、[31.2, 32.1)、[32.1, 35.5)、[35.5, 45.5]。进一步通过最小化风险函数确定最佳检测孕期，该时点位于19w+1至23w+6的范围内。为了量化误差对模型的影响，我们通过成功率函数曲线得出结论：低BMI组的曲线在阈值区段呈现较为陡峭的变化，意味着个体差异对整体情况的影响较小；而高BMI组则呈现相反的趋势，个体差异对其影响较为显著。

针对问题四，鉴于女胎结果均为健康，为便于建模，将问题转化为女胎染色体是否异常的二元判定，并构建基于X染色体Z值、GC含量、读段数等因素的两级联合概率判断模型。首先根据医学和生物学标准筛选样本高质量数据。然后采用两级联合判定法：第一阶段直接比对12、18、23号染色体Z值大小，根据临床阈值给出确定性概率；第二阶段分别训练逻辑回归和梯度提升树模型，为缓解概率输出的失配，采用Sigmoid与等渗回归进行校准，并平均融合两模型概率得到综合异常概率。随后通过最小化代价函数确定最优判定阈值。基于交叉验证，模型平均召回率0.82，精准率0.87。

关键词：广义混合效应模型 分位数回归树集成 梯度提升树 逻辑回归

一、 问题重述

1.1 问题背景

无创产前检测^[1] (NIPT) 通过采集孕妇外周血、检测血浆中胎儿游离 DNA, 以判断染色体是否存在数目异常。该方法无创、安全且灵敏度较高, 已成为重要的产前筛查手段。

影响 NIPT 准确性的关键因素包括孕周与孕妇体质指数 (BMI), 统一设定的检测时点难以兼顾个体差异^[2]。为此, 临床实践常依据 BMI 分组并给出差异化的建议检测时点, 以降低复检与延迟带来的风险; 进一步结合检测数据中的染色体 Z 值、GC 含量、测序读段与比对质量等指标, 可支持构建更为合理的分组与检测策略, 推动个性化产前筛查路径的形成。

1.2 问题提出

问题一: 由于 NIPT 结果准确性与 Y 染色体浓度有关, 试分析胎儿的 Y 染色体浓度与孕周数、BMI 等指标的相关特性, 建立二者之间的关系模型, 并检验其显著性。这一问题旨在揭示影响检测浓度变化的主要因素, 为后续优化检测时机提供依据。

问题二: 在仅使用孕妇 BMI 作为分组标准的前提下, 研究男胎 Y 染色体浓度达到阈值 (4%) 的最早时点。要求对孕妇群体进行合理分组, 给出不同 BMI 区间下 NIPT 的推荐检测时机, 并分析检测误差对结果的影响。

问题三: 考虑到 Y 染色体浓度可能受到多因素影响, 需要在模型中纳入孕妇身高、体重、年龄等附加指标, 综合分析这些变量对浓度变化的作用。进一步结合检测误差与胎儿 Y 染色体浓度达标比例的约束, 优化分组与检测时机, 使整体检测准确性和稳健性得到提升, 并分析检测误差对结果的影响。

问题四: 针对女胎, 由于无法利用 Y 染色体浓度, 需要结合染色体 13、18、21 的 Z 值及非整倍体标签, 建立异常判定模型^[3]。要求在综合考虑 GC 含量、测序读段及相关比例、BMI 等因素的基础上, 给出女胎异常的判定方法。

二、 问题分析

2.1 问题一

针对问题一, 本文采用多水平混合效应模型 (Logit-LMM) 来刻画 Y 染色体浓度与孕周、BMI 等指标间的关系。通过对原始数据进行标准化处理, 并引入孕周的 B 样条函数捕捉非线性趋势, 同时设置孕妇个体作为随机效应, 以刻画个体间差异。模型求解过程中, 结合似然估计与参数显著性检验, 获得了各指标对 Y 染色体浓度的作用方向与显著性水平, 为后续问题的变量筛选与规律挖掘提供了量化依据。

2.2 问题二

针对问题二, 本文采用分位数回归树模型对男胎孕妇的最佳 NIPT 检测孕周进行求解。首先按照 BMI 对样本进行分组, 构建分位数回归树以捕捉不同组别下孕周与检测成功率之间的非线性关系; 随后通过风险函数最小化方法确定各 BMI 组的风险最小孕周; 最后绘制成功率曲线并结合风险曲线, 推导出推荐的检测时点。该方法能够在分组差异下寻找检测时机的动态平衡, 兼顾风险控制与检测效率。

2.3 问题三

针对问题三，本文要求在不同 BMI 分组条件下，结合孕周与检测误差，确定男胎 Y 染色体浓度达到检测阈值的最优 NIPT 首检时点。通过建立 BMI 分组模型与分位数回归树方法，拟合孕周与首检成功率的关系，计算各组的风险最小孕周，并结合首检成功率 90% 的约束进行修正。进一步对检测误差的顺延性与波动性进行敏感性分析，从而给出合理的分组及对应的最佳检测时点，保证检测风险最小并具备临床可操作性。

2.4 问题四

针对问题四，本文结合逻辑回归与梯度提升树（GBDT）构建融合判别模型，用于判定女胎孕妇是否存在染色体异常。建模思路为：逻辑回归负责建模各特征（Z 值、GC 含量、读段比例、BMI 等）的线性可解释效应，GBDT 用于捕捉非线性和高阶交互关系。两模型输出的概率经双重校准后加权平均，并通过代价敏感阈值确定最终分类结果。求解过程中，采用交叉验证评估模型稳健性，并结合混淆矩阵与特征重要性分析验证模型的性能与可解释性，从而保证在降低漏检风险的同时保持较高的特异度。

三、 模型假设

- 1. 假设同一受检者内的多次测量相关，不同受检者之间近似独立。
- 2. 假设未观测的因素（如饮食、吸烟、药物）影响远小于孕周、BMI 等主效应。
- 3. 假设筛去采样点离群点后剩余成分均可视为准确值。

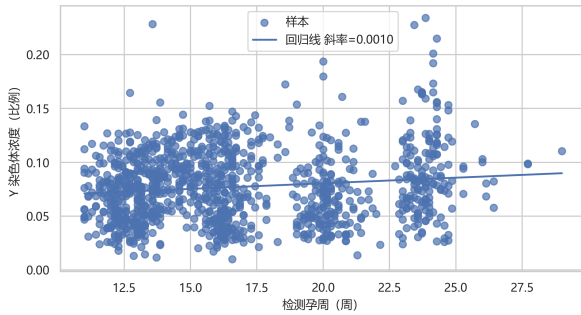
四、 符号说明

符号	意义	符号	意义
x_{day}	孕妇检测孕周日	x_{sum}	春、夏虚拟变量
x_{out}	春、秋虚拟变量	x_{win}	春、冬虚拟变量
y	Y 染色体浓度	PC_i	第 i 个主成分变量
x_{BMI}	孕妇 BMI 指标	t_{opt}	临时最佳检测孕周日
t_{opt}^*	最佳检测孕周日	C_{TIME}	时间代价权重
C_{TEST}	测试次数代价权重	C_{FAIL}	首检失败代价权重
n_{test}	检测次数	$\hat{\sigma}_i$	第 i 个个体计量误差
$s_{day,i}$	时间尺度	Δt_i	顺延量
t_{first}	修正后首检成功孕周日	$p_i(t)$	首次命中率
q	分位点数值	CG	CG 含量
ZN	N 号染色体的 Z 值	$P_{abnormal}$	女胎异常概率
N	样本总数	N_1	阳性样本数
N_2	阴性样本数	τ	概率阈值
FP	预测结果假阳性人数	FN	预测结果假阴性人数

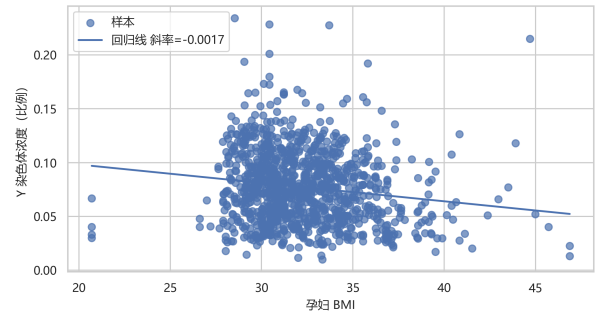
五、 数据分析

5.1 对指标关系的描述性分析

通过绘制 Y 染色体浓度分布的散点图，能够直观展示这些变量之间的关系，为后续的模型构建提供指示。以下是对孕妇孕周数和 BMI 指标的可视化分析结果：



(a) 孕周与 Y 染色体浓度关系图



(b) BMI 与 Y 染色体浓度关系图

图 1: 个体特征与 Y 染色体浓度关系

图 1(a) 和图 1(b) 表明孕周与 Y 染色体浓度之间未显示出显著相关性。因此，简单线性或非线性拟合无法揭示数据间潜在关系，后面采用其它合理模型进行分析。

5.2 Y 染色体浓度时序特征

此外，我们将孕妇检测时间按季节归类，通过绘制时间散点图，发现 y 染色体浓度和季节有潜在关系：

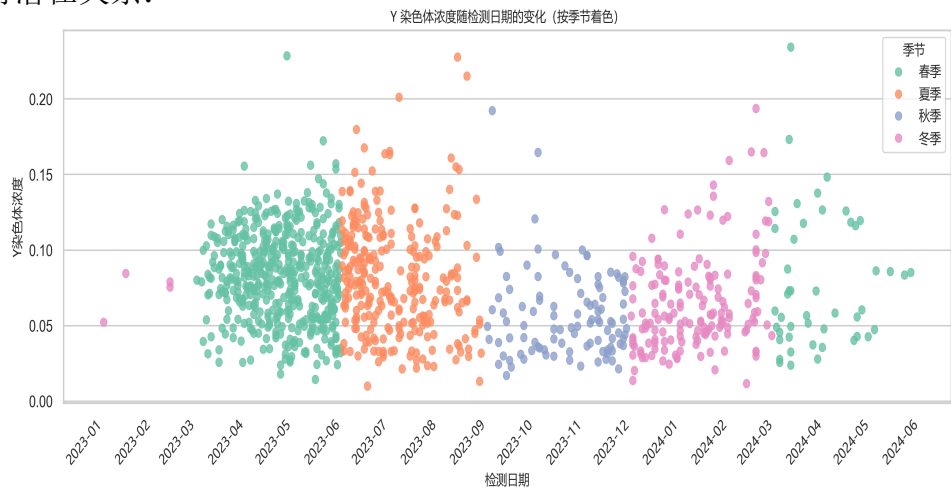


图 2: 季节性动态分布图

从图可知，y 染色体浓度在夏季较高，秋冬较低，可以将检测季节作为一个 y 染色体浓度的影响指标。

六、 问题一模型建立与求解

6.1 数据预处理

为了提升数据质量，同时方便后续模型建立，我们对数据进行如下处理：

6.1.1 剔除异常值

孕周、BMI 等生理指标受多种独立因素共同作用，根据中心极限定理，其总体分布在大样本人群中通常近似正态。因此，采用 3σ 原则识别潜在异常值，超过 3σ 的数据点视为离群点，识别出附件共有 50 组数据异常，进行剔除。

6.1.2 孕周标准化

在临床记录中，孕周常以“周 + 天”的形式表示，例如 $11w+6$ 表示妊娠 11 周零 6 天。为便于统计建模，把检测孕周统一格式转换为天，例如： $11w+6 = 11 \times 7 + 6 = 83$ 天。

6.1.3 季节指标的构建

为了进一步确定不同季节下 Y 染色体浓度的分布差异，画出箱线图：

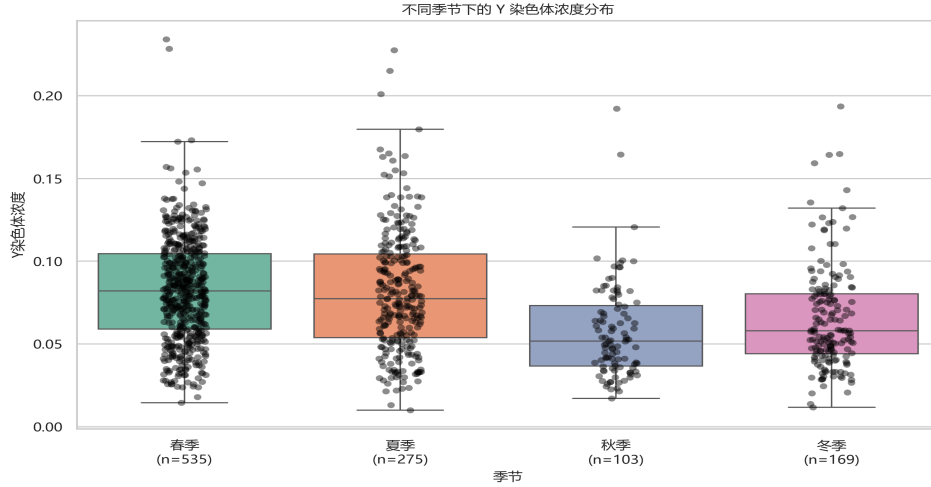


图 3: 不同季节下的 Y 染色体浓度分布图

可以确认四个季节的 Y 染色体浓度存在明显波动。因此，我们将检测周期转换为季节作为一个影响 Y 染色体浓度的指标，转换关系如下：

$$\begin{aligned} x_{sum} &= \begin{cases} 1, \text{month} = 6\ 7\ 8 \\ 0, \text{else} \end{cases} \\ x_{aut} &= \begin{cases} 1, \text{month} = 9\ 10\ 11 \\ 0, \text{else} \end{cases} \\ x_{win} &= \begin{cases} 1, \text{month} = 12\ 1\ 2 \\ 0, \text{else} \end{cases} \end{aligned} \quad (6.1)$$

6.1.4 指标的初步选取

首先可以由具体意义及医学表现确认以下指标可以不予考虑，简化模型：

- 1) 其它染色体 Z 值：主要用于非整倍体筛查，与研究目标不符，且可能混淆孕周与 BMI 的效应。
- 2) 其它浓度指标（如 X 染色体浓度）：与 Y 浓度存在强相关性，信息冗余，易引入多重共线性。
- 3) 染色体 GC 含量：整体可由全基因组 GC 含量代表，重复性强，建模价值有限。
- 4) 孕次、产次：更适用于群体风险建模，而非解释个体当前浓度，易引入混杂效应。

5) 胎儿健康状况：属于结局变量（outcome），不宜作为自变量纳入解释模型。

因此，过滤后需要进一步探讨的指标有：孕周、BMI、年龄、唯一比对读段数_log、原始读段数_log、比对比例、重复比例、过滤比例、GC。本文对其用 Pearson 相关判断与 Y 染色体是否有相关性。

6.2 相关性分析

为了初筛影响 Y 染色体浓度的重要指标，本文采用 Pearson 相关系数衡量变量间线性关系。考虑到部分变量与研究目标不符、存在强相关或属于结果指标（如 Z 值、染色体浓度、全基因组 GC 含量等），以及部分技术性指标已由其他变量体现，因此未予纳入分析。

最终选取孕周、BMI、年龄、唯一比对读段数、原始读段数、比对比例、重复比例及过滤这些指标，用以构建模型。另外在数据分析中探索发现季节对 Y 染色体浓度也有影响，所以本文也将其纳入模型，以确保结果的解释性和稳定性。

6.2.1 纵向数据的解释说明

由于每个孕妇有多次检测数据（纵向数据），不能把所有点当独立样本计算相关性。可先初步为后续模型探索指标，采用每个孕妇各项数据指标的均值进行初步分析处理。

6.2.2 Pearson 相关性分析

为系统评估各候选变量之间的相关性，采用非参数的 Spearman 秩相关系数进行检验。皮尔逊相关系数的计算公式为：

$$r_{XY} = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2} \sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2}} \quad (6.2)$$

其中， X_i, Y_i 分别表示第 i 个观测样本的取值， $\bar{X}, \bar{Y}$ 为各自的样本均值。皮尔逊相关系数 r_{XY} 的取值范围在 $[-1, 1]$ 之间，绝对值越接近 1，说明线性相关性越强；若接近 0，则表示两者之间几乎不存在线性关系。

通过热力图直观展示变量间的关系如下：

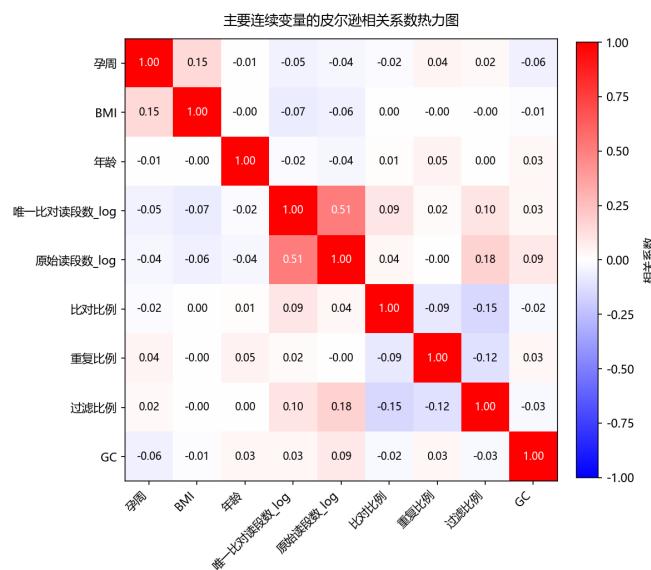


图 4: 主要连续变量的皮尔逊相关系数热力图

结果显示,唯一比对读段数与原始读段数存在中等强度的正相关 ($r=0.58$),其余质量指标之间仅存在弱相关性;孕周、BMI、年龄与技术指标相关性极低,基本独立。

基于此结果,我们在后续分析中采用主成分分析 (PCA) 对质量指标进行降维,以降低多重共线性对模型估计的影响。

6.3 Y 染色体浓度广义混合效应模型 (Logit-LMM)

6.3.1 浓度指标逻辑转换

在 NIPT 检测中,染色体浓度若直接使用读段数,会受到测序总量和实验条件差异的严重影响,缺乏样本间的可比性,因此首先需要将其转化为比例数据,以消除测序深度差异并统一量纲。公式为:

$$y' = \text{logit}(y) = \ln \left(\frac{y}{1-y} \right) \quad (6.3)$$

其中, y 表示 Y 染色体的浓度, y' 表示染色体片段占比与非片段占比的对数比值。

6.3.2 混合效应模型构建

在本问中, Y 染色体浓度属于典型的比例型数据需经过上述逻辑转化处理;同时,数据中同一孕妇往往有多次检测记录,存在个体内相关性,若忽略会导致显著性虚高和结果偏差,而线性混合效应模型 (LMM) 能够通过随机效应有效控制重复测量带来的依赖性。综合来看, Logit-LMM 既符合比例数据的分布特征,又能兼顾固定效应与孕妇个体差异。

设第 j 位孕妇在第 i 次检测中, Y 染色体浓度为 y_{ij} , 其基本模型可以表示为:

$$y'_{ij} = \text{logit}(y_{ij}) = \ln \left(\frac{y_{ij}}{1-y_{ij}} \right) = \beta_0 + \beta_1 X_{1,ij} + \beta_2 X_{2,ij} + \cdots + \beta_k X_{k,ij} + u_j + \varepsilon_{ij} \quad (6.4)$$

6.4 模型指标选择

6.4.1 标准化处理

由于不同变量的量纲差异很大,直接建模会导致协方差矩阵数值差异过大。通过 Z-score 标准化将各变量调整到均值为 0、标准差为 1 的统一尺度,可以提高数值计算的稳定性,且让回归系数具有可比性,更加直观比较不同变量对 Y 染色体浓度的相对影响,标准化公式为:

$$\tilde{X} = \frac{X - \mu}{\sigma} \quad (6.5)$$

在主成分分析之前进行标准化处理,可以保证每个指标在构建主成分时贡献均衡,更加客观地反映出各变量对 Y 染色体浓度的相关性。

6.4.2 主成分分析模型的建立

可知共有 n 个评价对象, m 个评价指标,指标分别为:测序深度、有效比对率、重复率。根据上述标准化处理后的数据,首先计算相关系数矩阵,相关系数矩阵 $R = (r_{ij})_{m \times m}$

$$r_{ij} = \frac{\sum_{k=1}^n \tilde{a}_{ki} \tilde{a}_{kj}}{n-1} \quad (6.6)$$

其中, $r_{ii} = 1, r_{ij} = r_{ji}, r_{ij}$ 为第 i 个指标和第 j 个指标的相关系数; $\tilde{a}_{ij}$ 表示第 i 个样本对象的第 j 个在经过标准化处理后的取值。

计算相关系数矩阵 R 的特征值和特征向量。计算得特征值

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_m \geq 0, \quad (6.7)$$

以及对应特征向量组 $w_1, w_2, \dots, w_n$, 其中

$$w_i = (w_{1i}, w_{2i}, \dots, w_{mi})^T. \quad (6.8)$$

由相应的特征向量组成 m 个新指标变量:

$$\begin{cases} y_1 = w_{11}\tilde{x}_1 + w_{21}\tilde{x}_2 + \cdots + w_{m1}\tilde{x}_m, \\ y_2 = w_{12}\tilde{x}_1 + w_{22}\tilde{x}_2 + \cdots + w_{m2}\tilde{x}_m, \\ \dots\dots\dots \\ y_m = w_{1m}\tilde{x}_1 + w_{2m}\tilde{x}_2 + \cdots + w_{mm}\tilde{x}_m, \end{cases} \quad (6.9)$$

其中, y_m 为第 m 主成分。

6.4.3 主成分变量的选择及分析

在主成分分析中, 我们保留特征值大于 1 的主成分, 并确保累计方差解释率超过 80%。为直观展示主成分对方差的解释能力, 绘制累积解释方差曲线图如下:

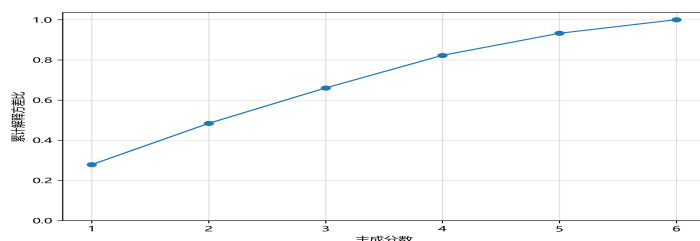


图 5: 累积解释方差图

从图中可以看出, 前四个主成分累计解释率已超过 80%, 说明它们足以涵盖绝大部分信息。因此, 后续分析中我们仅选择前四个主成分。

进一步地, 我们考察各原始变量在主成分上的载荷, 以便明确不同主成分所代表的潜在因子。下图展示了展示了各质量/技术指标在四个主成分上的贡献情况。

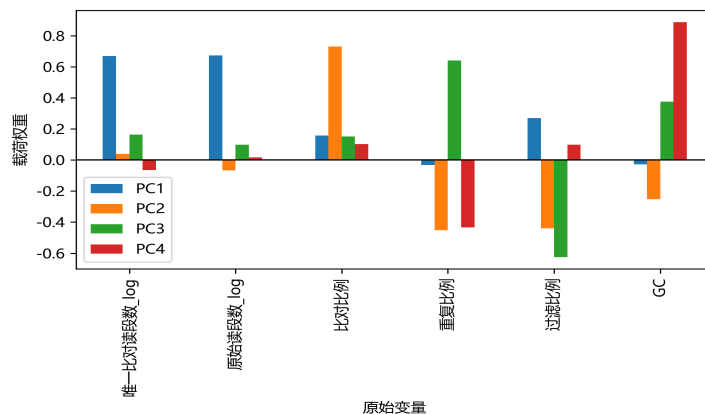


图 6: 主成分载荷分布图

从图中可以看出:

- **PC1**: 主要由原始读段数和唯一比对读段数贡献;
- **PC2**: 主要由比对比例贡献;
- **PC3**: 主要由重复率和过滤比例贡献;
- **PC4**: 主要由 GC 变量贡献。

基于此, PCA 把原始的 6 个测序质量指标压缩成 4 个主成分, 且每个主成分都有较好的解释性。至此, 主成分分析有效降低了变量维度, 提炼出关键质量信息, 并在此基础上缓解了多重共线性对模型估计的干扰。

综上, 模型最终拟合指标选择为: 孕周、BMI、年龄、季节、PC1 PC4。

6.5 样条修正的逻辑混合模型

为刻画孕周与 Y 染色体浓度之间的非线性关系, 我们将孕周作为分段样条函数纳入固定效应: 在若干结点 (knots) 处构造样条基函数, 使各段在结点处一阶连续并整体平滑。相应地, 模型中的孕周效应可表示为:

$$f(\text{孕周}) = \beta_0 + \sum_{k=1}^K \beta_k B_k + \varepsilon_i \quad (6.10)$$

其中, i 表示第 i 位孕妇在第 j 次检测的孕周; B_1 是第 1 个样条基函数; γ_1 是对应的回归系数。该处理避免了线性假设带来的偏差, 能够灵活捕捉早、中、晚孕期增长速率的差异, 并与随机效应共同建模个体间异质性与重复测量依赖。

6.6 Y 染色体浓度与各类指标关系模型

综上, 最终的关系模型为:

$$y_{ij} = \beta_0 + f(\text{孕周}) + \beta_1 X_{BMI,ij} + \beta_2 X_{age,ij} + \sum_{k=1}^3 \alpha_k X_{season,ijk} + \sum_{m=1}^M \delta_m PC_{m,ij} + u + \epsilon_{ij} \quad (6.11)$$

$$u = u_{0i} + u_{1i} \times \text{孕周} \quad (6.12)$$

其中, $PC_{m,ij}$ 表示主成分变量; u_{0i} 表示个体 i 的随机截距, u_{1i} 表示个体 i 的孕周随机斜率, 表示孕妇个体随孕周增长的个体异质性变化趋势。

6.7 显著性检验

在问题一中, 我们采用 logit-LMM 分析孕周数、BMI 等指标对 Y 染色体浓度的影响。为检验各固定效应参数的显著性, 本研究选用 **Wald 检验**。其优点是仅依赖于参数估计值及其标准误, 计算简便, 且在大样本下具有较好的渐近性质。

对于固定效应参数 β_k , Wald 统计量为:

$$W = \frac{\hat{\beta}_k}{SE(\hat{\beta}_k)}, \quad (6.13)$$

其中, $\hat{\beta}_k$ 为估计系数, $SE(\hat{\beta}_k)$ 为其标准误。在假设 $H_0: \beta_k = 0$ 下, W 近似服从标准正态分布。

若进行联合检验，则推广为：

$$W = (\hat{\beta} - \beta_0)^\top [\text{Var}(\hat{\beta})]^{-1} (\hat{\beta} - \beta_0), \quad (6.14)$$

在 H_0 下服从 χ^2 分布。

通过 Wald 检验可显著性地识别孕周数、BMI 等因素对 NIPT 检测时点的影响。

6.8 模型求解

6.8.1 模型求解结果展示

至此，已经建立了 Logit-LMM 模型的形式，并明确了自变量、固定效应和随机效应的设定。接下来，通过实证估计得到各变量的回归系数及其统计显著性，从而检验不同因素对 Y 染色体浓度的实际影响。结果如下：

表 1: 多水平混合效应模型估计结果（因变量：logit(Y 染色体浓度)）

变量	回归系数	标准误	P 值	95% 置信区间
常数项（截距）	-2.836	0.055	0.000	[-2.944,-2.728]
夏 vs 春	0.010	0.029	0.723	[-0.046,0.067]
秋 vs 春	-0.156	0.052	0.002	[-0.257,-0.055]
冬 vs 春	-0.212	0.046	0.000	[-0.302,-0.122]
孕周（样条 1）	0.206	0.074	0.005	[0.061,0.352]
孕周（样条 2）	0.299	0.066	0.000	[0.169,0.429]
孕周（样条 3）	0.538	0.129	0.000	[0.286,0.790]
孕周（样条 4）	1.533	0.166	0.000	[1.208,1.858]
BMI_z	-0.057	0.022	0.011	[-0.101,-0.013]
年龄_z	-0.041	0.026	0.117	[-0.091,0.010]
PC1	-0.021	0.007	0.002	[-0.034,-0.007]
PC2	0.024	0.008	0.001	[0.009,0.039]
PC3	0.018	0.008	0.022	[0.003,0.034]
PC4	0.005	0.008	0.575	[-0.012,0.021]
随机截距方差	0.161	—	—	—
随机斜率方差	0.022	—	—	—

该表展示了多水平混合效应模型对 logit 转换后的 Y 染色体浓度的回归结果。整体来看，孕周对 Y 浓度具有显著的非线性正向作用，秋季与冬季相较于春季浓度显著降低，BMI 对 Y 浓度存在显著负向影响，而年龄效应不显著。主成分中，PC1 显著负向、PC2 显著正向，提示存在潜在的系统性差异。随机效应结果表明不同孕妇间存在明显个体差异。总体结果验证了 Y 染色体浓度受孕周、季节及个体特征多因素共同影响的结论。

6.8.2 检验结果显著性

基于表一的多水平混合效应模型估计结果，对各变量的显著性进行检验，主要结论如下：

- **孕周效应显著：**孕周的分段样条变量均在 1% 水平上显著，说明 Y 染色体浓度随孕周的变化呈现出强烈的非线性趋势，是影响检测结果的关键因素。

- **季节效应部分显著**: 秋季与冬季相较于春季的回归系数均显著为负 ($P < 0.01$), 提示检测结果存在明显的季节性差异; 而夏季与春季之间无显著差异 ($P = 0.374$)。
- **BMI 的显著负效应**: BMI 对 Y 染色体浓度的影响在 5% 水平上显著 ($P = 0.028$), 回归系数为负, 表明 BMI 升高会降低 Y 染色体浓度水平。
- **年龄效应不显著**: 孕妇年龄在本模型中未通过显著性检验 ($P = 0.182$), 说明其对 Y 浓度的独立作用相对较弱。
- **主成分效应差异化**: PC1 显著负向 ($P < 0.01$), PC2 显著正向 ($P = 0.002$), 显示潜在的个体差异与实验条件对检测结果有系统性影响; PC3 与 PC4 未显著。
- **随机效应结果**: 随机截距和随机斜率的方差均为正, 提示不同孕妇之间存在显著的个体差异, 这支持了混合效应模型在该问题中的合理性。

6.9 模型结果分析

至此, 基于 Logit-LMM, 已给出对 Y 染色体浓度的影响因素的系数估计与显著性检验 (即上表), 定量刻画了孕周、BMI、年龄、季节以及测序质量主成分等变量的效应。为检验模型拟合质量并直观呈现关键效应, 本文进一步进行了模型结果的图形化分析。

6.9.1 残差拟合诊断

在具体效应分析前, 先对模型的总体拟合情况及方差齐性假设进行检验。通过绘制残差与拟合值的散点分布, 可以直观判断模型设定是否合理及残差方差是否均一, 从而评价 Logit-LMM 的稳健性。

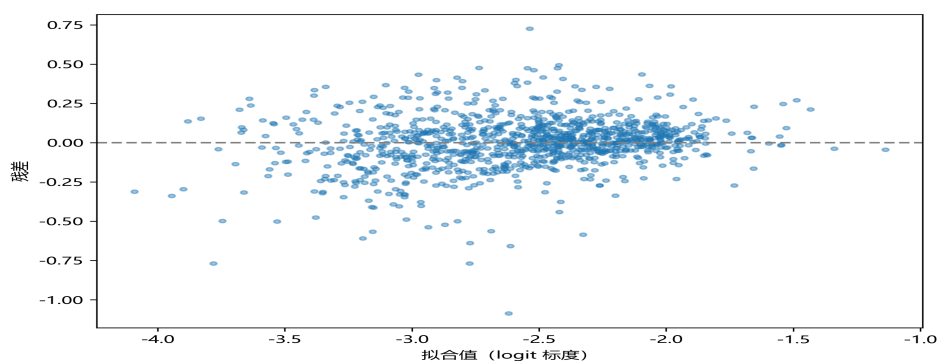
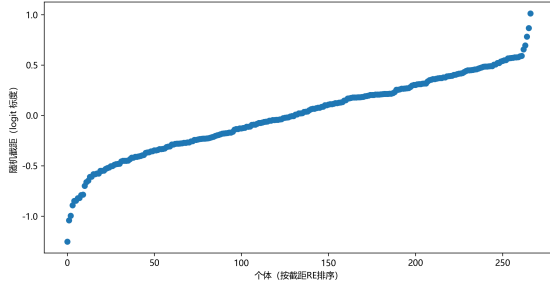


图 7: 残差拟合诊断图

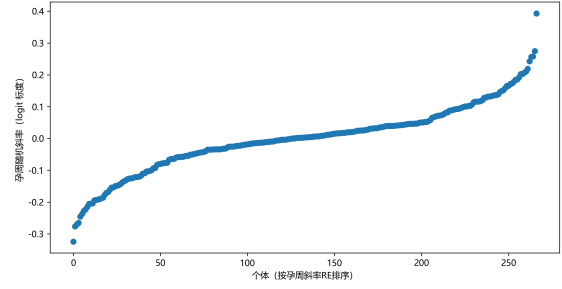
从图中可以看到, 残差点大致围绕零均匀分布, 未呈现系统性偏差, 也未见明显的漏斗状分布, 说明模型设定较为合理, 拟合无显著偏差, 方差齐性假设基本成立。

6.9.2 个体差异的可视化分析

为了进一步验证模型对个体差异的捕捉效果, 并直观呈现样本间的差异分布情况, 绘制了如下所示的两类毛毛虫图。



(a) 随机截距毛毛虫图



(b) 孕周斜率毛毛虫图

图 8: 随机截距与斜率的个体差异特征图

从随机截距图可以看出，个体间的基线差异呈现出连续分布，而非集中在少数异常值；孕周斜率图则表明个体随孕周变化的趋势同样表现为广泛存在的差异，而不是离散的极端点。两图共同说明：个体内相关被模型充分捕捉，差异性分布具有普遍性和连续性，从而为模型的合理性与稳定性提供了可视化证据。

综上，本节对模型结果进行了系统分析。通过残差分析与随机效应分析，结果表明个体差异广泛存在且呈连续分布，随机截距与孕周斜率的毛毛虫图直观展示了个体水平上的变异性，证明模型有效捕捉到了个体内相关结构。模型结果分析不仅验证了模型设定的合理性与稳健性，为后续分组策略与最佳检测时点的确定提供了坚实依据。

七、 问题二的模型建立与求解

基于问题二中“男胎孕妇的 BMI 是影响胎儿 Y 染色体浓度最早达标时间的主要因素”的研究要求，本文构建了以 BMI 合理分组、基于经验分位数的最佳 NIPT 时点选择模型为核心的模型求解过程，解决了如何在不同 BMI 区间下合理确定最佳 NIPT 时点，并进一步分析检测误差对结果的影响。为更直观展示模型的构建与求解步骤，本文设计了针对问题二的模型建立与求解流程图，如下所示：

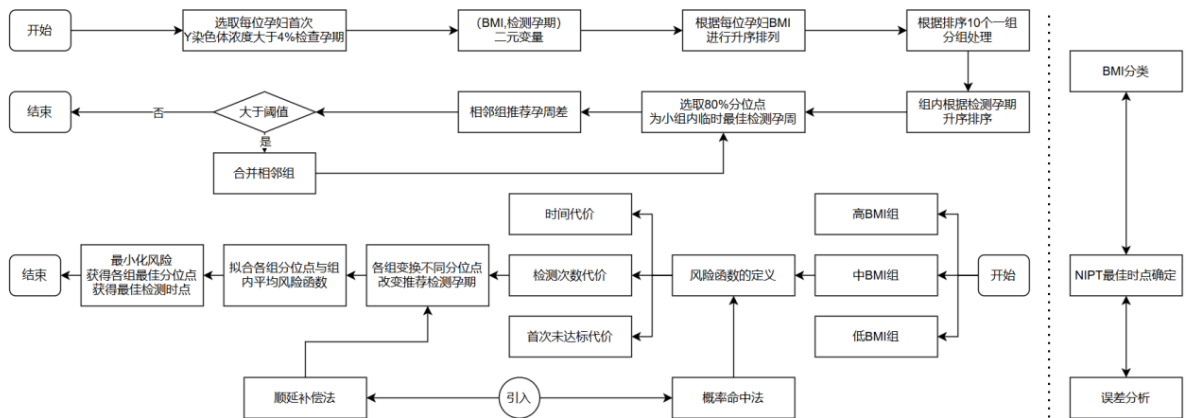


图 9: 问题二技术路线图

7.1 基于经验分位数的最佳 NIPT 时点选择模型的建立

针对本问题中，根据 BMI 指数与 Y 染色体浓度高度相关特征，对不同孕妇的 BMI 指数，借助其 Y 染色体浓度的最早达标不同时间进行分组处理。便于管理并有助于不同孕妇最佳 NIPT 时点的确定。

7.1.1 分位数法

分位数法完全基于数据本身来决定分组和推荐时点，避免了人为设定分组区间的主观性，能够更加客观地反映样本的真实分布。特别是对于 BMI 这种连续变量，采用分位数法能够自动地根据样本分布划分区间，从而确保每个组的样本量尽量且平衡，避免了过小或过大的组间差异。

其次，可以通过调整分位数（如 30%, 40% 分位数等）可以控制推荐时点的保守程度，灵活应对不同的风险要求。在实际操作中，如果某些组的推荐时点接近风险区间（如 28 周以后），可以适当前移，进一步优化推荐时点的选择。

7.1.2 分类模型的建立

Step1: 针对不同检测孕期的 Y 染色体浓度孕妇样本，采取对检测孕期进行升序处理找到第一次满足 $Y \geq 4\%$ 的孕周数据 x_{day} ，并对未找到 $Y \geq 4\%$ 的孕妇样本进行剔除，得到一组二元数据 (T_{first}, x_{BMI}) 。

Step2: 由于每个孕妇有多次检测记录，用其 BMI 的中位数做代表，将样本按 BMI 从小到大排序做等频分箱，平均分为 K 组 ($K=10$)，得到分组样本 $G_1, G_2, \dots, G_n$ 。（遇到重复值自动降组）

Step3: 由于检测成本和时间收益的存在，选取稳妥经验分位数 80% 作为该组的临时推荐孕周日长：

$$t_{opt,j} = Q_q(\{T_{first,i}^* : i \in G_j\}) \quad (7.1)$$

其中， Q_q 表示分位数函数； q 是所选的分位数，即 $q = 80\%$ ； $x_{day,i}^*$ 表示第 i 位孕妇的 Y 染色体浓度最早达标孕周。

Step4: 合并相邻项，比较相邻两组的推荐孕周日差：

$$|t_{opt,j} - t_{opt,j+1}| \leq \delta \quad (7.2)$$

若 $\delta \leq 3.5$ ，则合并为一个更大的 BMI 区间；合并后对新组重新算分位数与区间；迭代执行，直到所有相邻组的推荐检测孕周日差异都大于 δ 则完成分组。

7.1.3 不同组别推荐检测孕期的确定

由于 BMI 差异导致不同孕妇推荐检测孕期存在差距，由第一问相关性分析可知：BMI 与 Y 染色体浓度存在负相关特性，因此 BMI 组的推荐查孕期往往高于低 BMI 组推荐检测孕期。

问题中存在检测孕期异常时间分为早期发现、中期发现和晚期发现。该类风险对不同 BMI 程度的组别明显呈现在不同风险程度。因此不同 BMI 组的孕妇应选取不同的分位点来确定最佳 NIPT 时点。同时，过早检测会由于 Y 染色体浓度未达标而产生重复测风险；过晚检测会导致治疗成功率下降的风险。

为了量化不同检测时间下，时间延长导致治疗成功率风险的影响，再次的检测的成本损失风险影响，本文引用风险评估函数进行表示：

1) 时间风险（延迟确诊代价）

$$R_{time} = \lambda \cdot E[t_{opt,i}^*] \quad (7.3)$$

其中， λ 定义为分段函数：

$$\lambda = \begin{cases} c_1, & t_{opt,i}^* \leq 84 \\ c_2, & 84 < t_{opt,i}^* \leq 189 \\ c_3, & 189 < t_{opt,i}^* \end{cases} \quad (7.4)$$

其中，如果首检成功，确认孕期 $t_{\text{opt},i}^* = t_{\text{opt},i}$ ；如果首检不成功，孕期 $t_{\text{opt},i}^* = t_{\text{opt},i} + \Delta$ 。
 $c_1 < c_2 < c_3$ 反映不同时期的风险状况不同。

2) 检测次数代价

每做一次 NIPT，都存在成本，该成本不仅在于检测费用，还包括人力和心理负担等。为了简化模型，在这两阶段下进行合理假设：若第一次检测时 Y 染色体的浓度小于 4% 则会进行重测，且第二次测试一定成功，则：

$$R_{\text{test}} = C_{\text{TEST}} \cdot E[n_{\text{test}}] \quad (7.5)$$

3) 首次未达标代价

即便后续复检能补救，但首检失败会带来额外的“惩罚”，例如孕妇焦虑、沟通成本、延迟确诊的潜在医疗风险等，则有：

$$R_{\text{fail}} = C_{\text{FAIL}} \cdot (1 - p_{\text{success}}) \quad (7.6)$$

其中， C_{FAIL} 表示首检失败一次的代价； p_{success} 组内个体首检成功率，这里使用样本概率代替个体概率。

3) 总期望风险

综上，完整的期望风险框架为：

$$R_{\text{total}}(\tau) = R_{\text{time}} + R_{\text{test}} + R_{\text{fail}} \quad (7.7)$$

7.1.4 确定最佳 NIPT 时点

至此，已完成对男胎孕妇的 BMI 进行合理分组，并能够得到每组的 BMI 区间，每组都有对应的“孕周分布”数据。

在每一组里，可以选取不同的分位数（比如 50%、60%、70% ……），每一个分位数都会对应一个推荐检测孕周。但是，不同分位数对应的检测时点会带来不同的风险：

- 分位数太低 → 检测过早 → 复检概率大（检测成本高）；
- 分位数太高 → 检测过晚 → 延误诊断（时间风险大）。

因此，需要在每一组上，评估不同分位数下的风险函数。在风险评估结果中，总会有一个分位点能让总风险函数最小。这个分位点就是“最佳分位点”，即每组孕妇推荐的最佳检测孕周（NIPT 时点）

7.2 基于经验分位数的最佳 NIPT 时点选择模型的求解

7.2.1 BMI 合理分组区间确定

基于分位数法建立的 BMI 分类模型，先剔除已满足 $y \geq 4\%$ 的孕周数据，再将孕妇按 BMI 排序分组，并取各组孕周的 80% 分位数作为推荐检测点，相邻组差值过小则合并。最终得到男胎孕妇的 BMI 合理分组区间及每个区间样本数量统计如下图：

表 2: BMI 合理分组区间

BMI 区间	[20.7,30.9)	[30.9,31.6)	[31.6,35.9)	[35.9,45.0)
样本数量	104	26	104	26

7.2.2 BMI 各分组区间最佳 NIPT 时点确定

为评估不同分位数下的总体风险水平，并据此为 BMI 各分组区间选择最优分位数，即使总风险最小的点，绘制了 BMI 各分组区间总风险-分位数曲线如下：

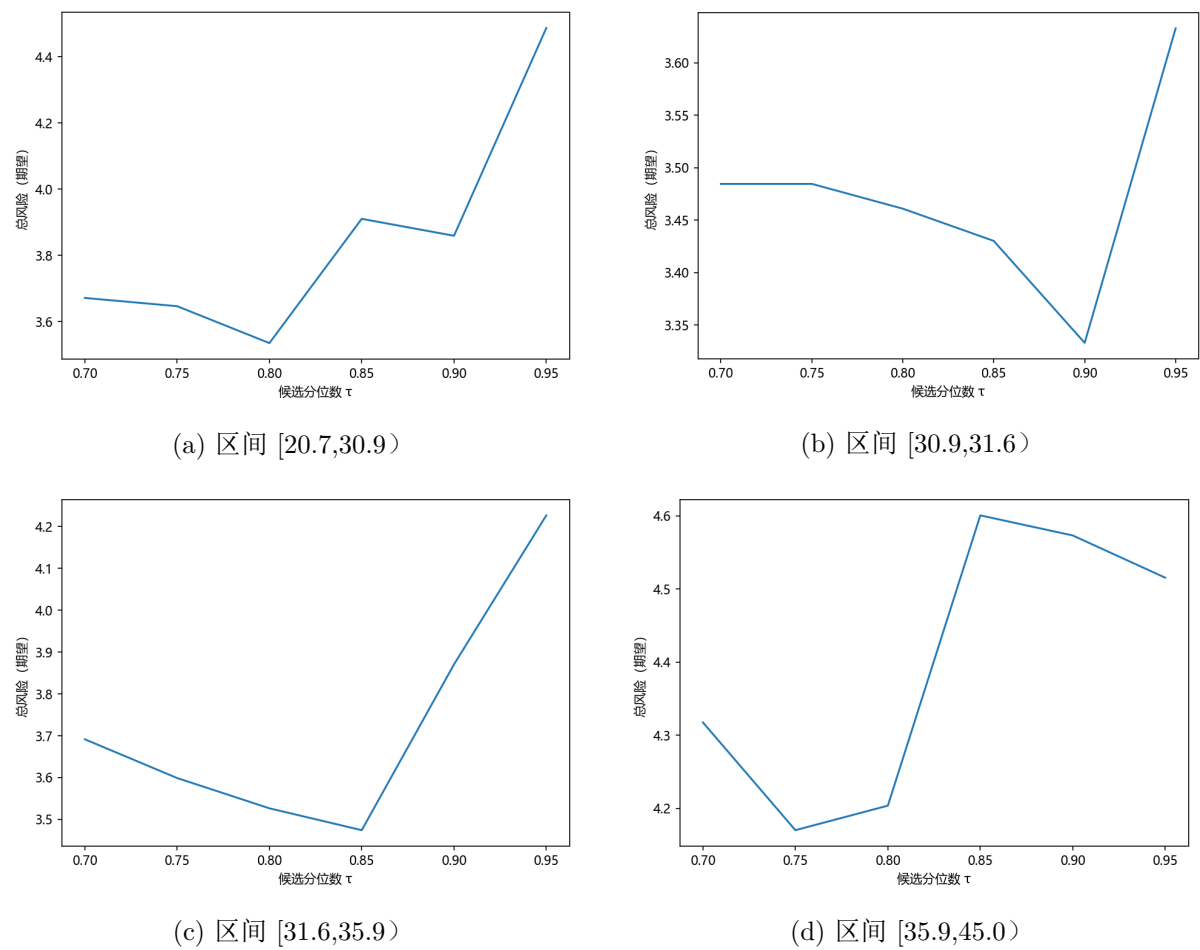


图 10: BMI 各分组区间总风险-分位数曲线图

根据图片，选择使风险最小的分位数作为 BMI 各分组区间的推荐检测时点，得到每个 BMI 分组的最佳 NIPT 时点如下：

表 3: BMI 合理分组区间及最佳 NIPT 时点

BMI 区间	[20.7,30.9)	[30.9,31.6)	[31.6,35.9)	[35.9,45.0)
最优分位数	0.80	0.90	0.85	0.75
最佳 NIPT 时点 (周 + 天)	13w+6	13w+6	13w+6	16w+3

7.3 检测误差对结果的影响模型的建立

在问题二中，实际检测的 Y 染色体浓度易受多方面因素影响而产生误差：如测序深度和扩增效率带来的技术噪声，BMI、孕周和胎盘 cDNA 释放差异造成的生物学波动，以及样本保存、运输等额外变量。这些因素会导致观测值与真实浓度偏离，特别是当判定在 4% 阈值附近时可能出现假阴性。因此需要对这些误差进行分析，研究其影响。

本文采用顺延补偿法和概率命中法对误差影响进行分析。

7.3.1 顺延补偿法

该方法通过将测量误差转化为时间上的不确定性，确保检测时点的准确性。顺延补偿法的核心思想是：如果某个样本的观测 Y 染色体浓度偏低，可能导致其未能在推荐孕周达标，从而进入了复检的顺延过程。

个体计量噪声估计：

$$\sigma_i = \sqrt{\frac{\hat{p}_i(1 - \hat{p}_i)}{n_i}} \quad (7.8)$$

其中， $\hat{p}_i$ 为第 i 个样本在首测孕周时的 Y 染色体浓度， n_i 为第 i 个样本下首检成功时侯浓度下的原始读段数。

浓度变化速率： 由于 Y 浓度通常随着孕周变化，本文可以通过拟合 Y 染色体浓度与孕周的线性关系来得到浓度随时间的变化速率，求导可得浓度随时间变化速率：

$$v = \frac{d\hat{p}}{dt}. \quad (7.9)$$

时间尺度： 时间尺度 $s_{day,i}$ 表示第 i 个样本的噪声对时间的影响，这个参数决定了每个样本的误差在时间上的表现。时间误差越大，顺延量就越大：

$$s_{day,i} = \frac{\sigma_i}{d\hat{p}_i/dt}. \quad (7.10)$$

顺延量： 顺延量是基于时间尺度和置信因子 z 计算的，确保误差对时点的影响能够被合理补偿：

$$\Delta t_i = z \cdot s_{day,i}. \quad (7.11)$$

其中， z 为置信因子，其通过设定误差的容忍度来影响检测时点的推迟程度，通常取值为 1.2816（对应 90% 置信度）。

首检成功时间点的修正：

$$T_{first}^* = T_{first} + \Delta t_i \quad (7.12)$$

7.3.2 命中概率法

我们可以认为所检测的 Y 染色体浓度的不一定等于真实值，存在波动 $\varepsilon \sim N(0, \sigma^2)$ 。因此浓度是否超过阈值并不是确定事件，而是一个概率事件。

采用给定推荐检测孕周时点 t ，计算个体在该时点“首检成功”的概率的方式：

$$p_i(t) = P(\hat{Y}_i(t) \geq 0.04) = \Phi\left(\frac{\mu_i(t) - 0.04}{\sigma_i}\right) \quad (7.13)$$

其中， $\mu_i(t)$ 是孕周 t 时的预计 Y 染色体浓度，通过 Y 染色体浓度与孕周的线性回归拟合获得。 σ_i 是每一个样本的检测噪声，假定正态分布如图公式， $p_i(t)$ 是在时点 t ，浓度大于 0.04 的概率。

风险函数的修正

1) 首检命中率：

$$\bar{p}(t) = \frac{1}{n} \sum_i p_i(t) \quad (7.14)$$

2) 期望确认孕周:

$$E[t_{opt,i}^*] = \frac{1}{n} \sum_i [p_i(t) \cdot t + (1 - p_i(t)) \cdot (t + \Delta)] \quad (7.15)$$

3) 期望检测周期:

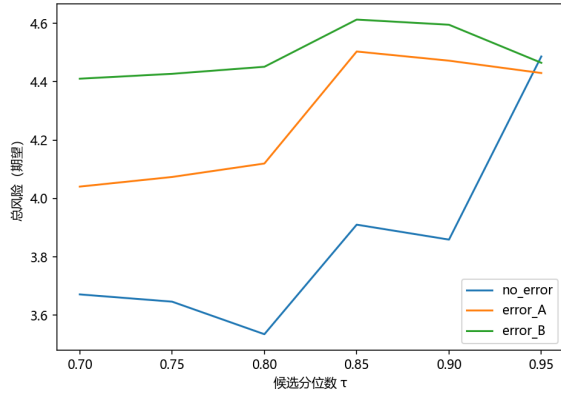
$$E[n_{test}] = \frac{1}{n} \sum_i [p_i(t) \cdot 1 + (1 - p_i(t)) \cdot 2] \quad (7.16)$$

4) 首检未达惩罚:

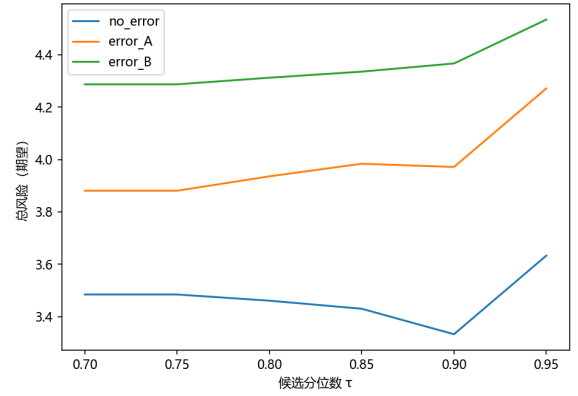
$$C_{FAIL} \cdot (1 - \bar{p}(t)) \quad (7.17)$$

7.4 检测误差对结果的影响模型的求解

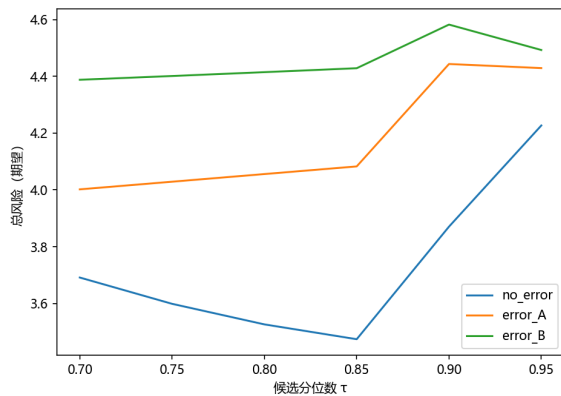
基于该模型, 本节进一步对不同 BMI 分组区间进行实证求解, 通过绘制“总风险-分位数曲线”对比不同误差假设下的结果, 评估检测误差对最优分位数选择与风险水平的影响。



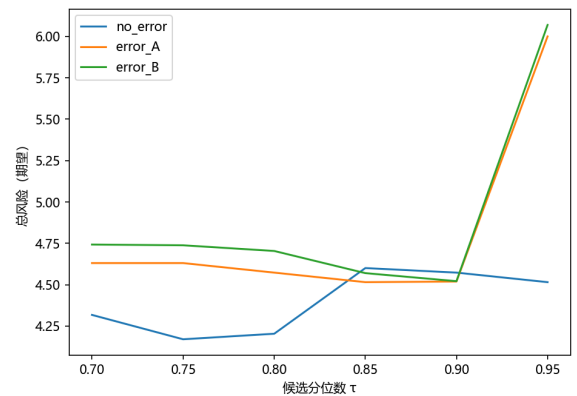
(a) 区间 [20.7, 30.9)



(b) 区间 [30.9, 31.6)



(c) 区间 [31.6, 35.9)



(d) 区间 [35.9, 45.0)

图 11: 加入检测误差后 BMI 各分组区间总风险-分位数曲线图

图中 error_A 表示顺延补偿型误差情境, error_B 表示概率波动型误差情境, no_error 表示无误差情境。从图中可以看出:

- **总体趋势:** 无论哪一个 BMI 区间, 引入检测误差都会显著抬高总风险曲线, 并使

得风险最优分位数向左（较小分位点）偏移。这说明检测误差使得较早的检测更具稳健性，以降低首检失败及重复检测带来的额外风险。

- **区间差异：**低 BMI 区间（[20.7, 30.9）、[30.9, 31.6]）的误差敏感性相对较小，虽然风险曲线整体上移，但最优分位数位置变化有限；而在高 BMI 区间（[31.6, 35.9）、[35.9, 45.0]），误差导致的风险抬升更为明显，且曲线在高分位数区域急剧上升，说明高 BMI 孕妇群体对检测误差更为敏感。
- **模型差异：**在三种情景下，error_B 模型通常带来最高风险，其次是 error_A，no_error 风险最低。这与模型设定中对误差传播路径及累积效应的不同假设相一致。

为进一步直观呈现不同 BMI 区间下的具体结果，本文对顺延补偿型误差情境、概率波动型误差情境下及无误差情境下的各指标进行了统计与对比。下面三张表展示了三种情境下不同 BMI 区间对应的最优分位数选择、首检命中率水平、期望确认孕周以及最终的总风险值。

表 4: 不同 BMI 区间在顺延补偿型误差情境下的最优检测策略结果

BMI 区间	最优分位数	首检命中率	期望确认孕周（周 + 天）	总风险
[20.7,30.9)	0.7	0.9231	17w+5	4.0407
[30.9,31.6)	0.7	0.9615	17w+1	3.8808
[31.6,35.9)	0.7	0.9423	17w+4	4.0019
[35.9,45.0)	0.85	0.9231	24w+0	4.5154

表 5: 不同 BMI 区间在概率波动型误差情境下的最优检测策略结果

BMI 区间	最优分位数	首检命中率	期望确认孕周（周 + 天）	总风险
[20.7,30.9)	0.7	0.7502	18w+1	4.4104
[30.9,31.6)	0.7	0.7681	17w+5	4.2870
[31.6,35.9)	0.7	0.7609	18w+1	4.3877
[35.9,45.0)	0.9	0.9191	25w+4	4.5214

表 6: 不同 BMI 区间在无误差情境下的最优检测策略结果

BMI 区间	最优分位数	首检命中率	期望确认孕周（周 + 天）	总风险
[20.7,30.9)	0.8	0.8269	14w+3	3.5349
[30.9,31.6)	0.9	0.9231	14w+1	3.3330
[31.6,35.9)	0.85	0.8558	14w+2	3.4743
[35.9,45.0)	0.75	0.7692	17w+1	4.1703

八、 问题三模型的建立与求解

8.1 分位数回归树集成模型的建立

8.1.1 分位数回归树集成模型

问题考虑在第二问的基础上，引入身高、体重等多种指标因素对 Y 染色体浓度的影响，对孕妇的 BMI 值进行分类处理。分位数和回归树均为预测模型的基本算法。本文将采用分位数回归树集成模型^[4]来构建分类体系，旨在实现检测时点的优化选择。

分位数回归方法^[4]是一种直接预测变量分布不同分位点下取值的模型，它通过对条件分布进行建模来获得更加全面的信息，能够更好地反映数据的整体分布情况。相比传统均值回归，分位数回归不仅能揭示均值水平，还能揭示不同分布位置下的特征效应，因而更加稳健，尤其适用于存在异方差性或异常值的数据。

回归树模型具有非参数建模能力，能够自动捕捉非线性和变量间的复杂交互，适合高维建模。分布在不同分位点下的分位数回归树能够在树的分裂节点选择最优切分变量和位置，用于预测目标分布的不同分位数。通过集成多棵树模型，可以有效降低过拟合风险，提高预测精度。

因此采用分位数回归树集成方式，它在回归树的基础上引入分位损失函数，直接建模 $q = 0.1, 0.25, 0.5, 0.9$ 等分位点，利用不同分位数的信息构建整体模型，从而更全面刻画 Y 染色体浓度分布特征，提升模型对检测时点推荐的鲁棒性。该模型能够反映出不同 BMI 区间下孕妇的 Y 染色体浓度变化规律，进而得到更科学合理的检测推荐方案。

8.1.2 模型的建立

1) 模型输入指标的选择

根据问题一中降维后的 Logit-LMM 模型包含参数指标与 Y 染色体浓度的显著性关系，选择身高、体重、季节、PC1（测序深度）、PC2（比对效率）、PC4（GC 偏倚因子）等变量作为输入变量构建分类模型。

采用第二问中顺延补偿法对样本 NIPT 首检成功孕周期进行修正：

$$t = \hat{t} + \Delta t_i \quad (8.1)$$

其中， $\hat{t}$ 为首次达标孕周日的修正值（实际真实值）；样本中首次达到孕周日的数值， Δt_i 表示顺延量。

2) 分类核心优化指标（分位数损失函数）

$$\ell_\tau(t, \hat{t}) = \begin{cases} \tau \cdot (t - \hat{t}), & t \geq \hat{t}, \\ (\tau - 1) \cdot (t - \hat{t}), & t < \hat{t}, \end{cases} \quad (8.2)$$

其中， $\hat{t}$ 为首次达标孕周日的真实值， $\hat{t}$ 为预测值， τ 为目标分位数。为考虑风险性特征， τ 一般取 0.8, 0.9。

$$\min_f \frac{1}{n} \sum_{i=1}^n \ell_\tau(T_i, f(X_i)), \quad (8.3)$$

其中， f 为学习函数，这里为分位数梯度提升树构成的预测函数。通过该公式可将回归树回归问题转变成损失目标，转化为分位数损失求解，确保输出结果满足临床和检测的稳健性要求。

3) 逐步加法逼近真实分位数条件函数

$$f_0(x) = \text{Quantile}_\tau(\{T_i\}), \quad (8.4)$$

$$f_m(x) = f_{m-1}(x) + \nu h_m(x), \quad m = 1, \dots, M, \quad (8.5)$$

其中, $f_0(x)$ 为初始化模型, 取所有训练样本 τ 分位数作为整体初始预测值; $f_m(x)$ 为第 m 次迭代的预测函数; $h_m(x)$ 为残差拟合得到的第 m 棵回归树; ν 为学习率 ($\nu \in (0, 1]$)。

8.2 分析检测误差模型的建立

8.2.1 内生式误差处理

在引入顺延补偿性修正时间点, 模型虽然能够降低因实验波动带来的孕妇失败风险, 但仍然存在内生性误差。其原因主要在于: 不同孕妇之间的生理差异、测序质量差异无法完全通过统一的检验手段来校正。这些差异会导致相同孕周下的胎儿 DNA 含量分布出现系统性偏移, 从而使得临床推荐时点在部分个体中仍可能偏早或偏晚。

8.2.2 成功概率曲线的引用

需要预测的量是孕妇在首检时能否达标的孕周 T 。如果直接用普通回归, 模型只能给出一个点预测, 但无法反映测量误差和个体差异。因此这里采用分位数回归的方式量化:

$$P(T \leq Q_{T|X}(q)|X) = q \quad (8.6)$$

其中, $Q_{T|X}(q)$ 就是在特征 X 下, 孕周 T 的第 q 分位点。则可获得下面区间:

$$[Q_{T|X}(0.3), Q_{T|X}(0.7)]$$

表示大约有 40% 的孕妇 (70%-30%) 的首次达标孕周落在这个范围内。区间越窄可以反映模型越确定, 检测噪声小、个体差异小; 反之区间越宽则不确定性越大, 说明个体差异或测序噪声影响更显著。

进一步定义成功概率为:

$$\Pi(t) = P(T \leq t) = \sup\{q \in [0, 1] : Q(q) \leq t\} \quad (8.7)$$

借助上述分位数回归曲线可以找到在某个水平下, 有多少 q 的 $Q(q)$ 不超过 t , 得到每个样本的成功概率。

对每个样本的分位数序列 (q_i, Q_i) , 用线性插值法求出任意 t 时的 $\Pi(t)$ 。

通过对每个 BMI 小组内个体的 $\Pi(t)$ 进行均值处理得到该小组的成功率曲线。这条曲线体现如果在孕周 t 检测, 有多少比例的孕妇已经达标, 借助其陡峭程度可以体现组内的不确定性大小, 即由于个体误差, 导致的误差来源不同。

定义组内平均成功率为:

$$\Pi_g(t) = \frac{1}{n_g} \sum_{i=1}^{n_g} \Pi_i(t) \quad (8.8)$$

可知基于成功率函数修正风险函数结果为:

$$\begin{cases} R_{time} = C_{TIME} [\Pi_g(t) \cdot t + (1 - \Pi_g(t)) \cdot (t + \Delta)] \\ R_{test} = C_{TEST} [\Pi_g(t) \cdot 1 + (1 - \Pi_g(t)) \cdot 2] \\ R_{fail} = C_{FAIL} (1 - \Pi_g(t)) \\ R_{total}(\tau) = R_{time} + R_{test} + R_{fail} \end{cases} \quad (8.9)$$

8.3 基于分位数回归树的最佳 NIPT 时点模型的求解

8.3.1 分位数回归树集成模型算法设计

模型的迭代更新算法伪代码如下：

Algorithm 1: 基于分位数梯度提升的迭代更新 (τ -GBDT)

输入: 训练样本 $\{(X_i, T_i)\}_{i=1}^N$; 学习率 $\nu \in (0, 1]$; 分位数 $\tau \in (0, 1)$; 最大迭代轮数 M

输出: 加性模型 $f_M(\cdot)$

// 初始化: 常数模型 (可取经验分位或最小化经验损失的常数)

设 $f_0(x) \leftarrow \arg \min_c \sum_{i=1}^N \ell(T_i, c)$;

for $m = 1$ **to** M **do**

 // Step1 伪残差计算

for $i = 1$ **to** N **do**

$g_i \leftarrow \tau - \mathbf{1}\{T_i < f_{m-1}(X_i)\}$;

 // Step2 回归树的拟合

 用回归树 $h_m(\cdot)$ 拟合样本 $\{(X_i, g_i)\}_{i=1}^N$, 得到叶结点划分 $\{R_{m\ell}\}_{\ell=1}^{L_m}$;

 // Step3 计算叶节点步长

for $\ell = 1$ **to** L_m **do**

$\gamma_{m\ell} \leftarrow \text{Quantile}_{\tau}\left(\{T_i - f_{m-1}(X_i) : X_i \in R_{m\ell}\}\right)$;

 // Step4 模型更新

$f_m(x) \leftarrow f_{m-1}(x) + \nu \sum_{\ell=1}^{L_m} \gamma_{m\ell} \mathbf{1}\{x \in R_{m\ell}\}$;

8.3.2 最佳 NIPT 时点的求解结果及分析

在完成分位数回归树集成模型的建立之后, 首先需要根据该方法对男胎孕妇的 BMI 值进行合理分组。分组沿用问题二的方法, 得到 BMI 合理分组区间如下:

表 7: 考虑多种因素下的 BMI 合理分组区间

BMI 区间	[20.7,30.0)	[30.0,30.5)	[30.5,31.2)	[31.2,32.1)	[32.1,35.5)	[35.5,45.0)
样本数量	76	27	28	25	78	26

由于 BMI 会系统性影响 Y 浓度上升的速度与波动, 从而改变首检成功概率曲线, 不同 BMI 组的首检成功概率曲线及其增长斜率不同, 为此, 必须在每一个 BMI 组内分别求解最优孕周点, 以确保期望风险最小化的计算结果更加准确和可靠。故而必须在每个 BMI 组内单独求解使期望风险最小的孕周。如得到某典型组风险曲线图如下:

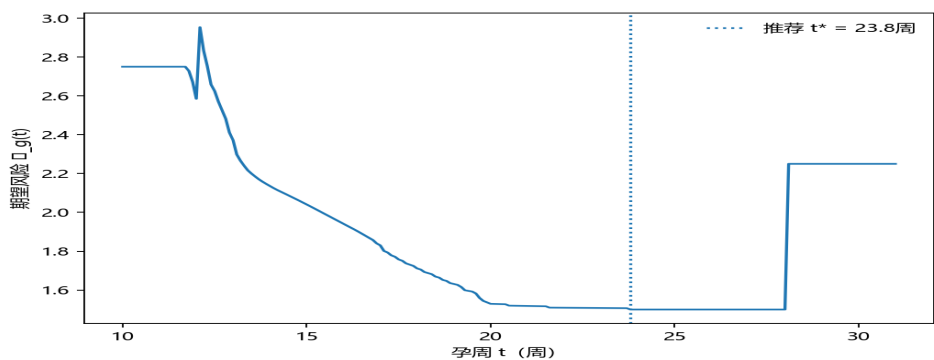


图 12: 某典型组风险曲线图

该图展示了典型组 Group 5 的风险曲线，整体呈现“U 型”特征：孕早期因首检成功率偏低风险较高，中期随成功率提升风险逐渐下降并在约 23.8 周达到最低，（即该典型组风险最小孕周为 23.8 周）。

对全部 BMI 组重复上述过程，可获得每一个 BMI 分组区间的风险最小孕周如下，风险最小孕周综合权衡时间风险、复检风险和失败惩罚，所以风险最小孕周即每组的最佳 NIPT 时点。统计每一个 BMI 分组区间的最佳 NIPT 时点如下：

表 8: 考虑多种因素下的 BMI 合理分组区间及最佳 NIPT 时点

BMI 区间	[20.7,30.0)	[30.0,30.5)	[30.5,31.2)	[31.2,32.1)	[32.1,35.5)	[35.5,45.0)
推荐孕周	20	20	19.2	20.6	21.1	23.8
成功概率	0.905	0.900	0.902	0.901	0.904	0.903

8.3.3 分析检测误差对结果的影响

在问题三中，本文重点考察检测误差对最佳 NIPT 检测时点选择的影响。从理论建模的角度看，检测误差主要通过增加测序数据的波动性（方差）改变首检成功率曲线的形态。当误差较小时，曲线在阈值附近陡峭上升，容易在某一临界孕周点实现从失败到成功的快速转变，此时风险最优检测时点与“首检成功率达到 90% 的时点”基本一致。而当误差较大时，曲线在阈值附近变得平缓，导致模型为了降低复检风险和首检失败惩罚，会倾向于将最佳 NIPT 检测时点推向更早的孕周。

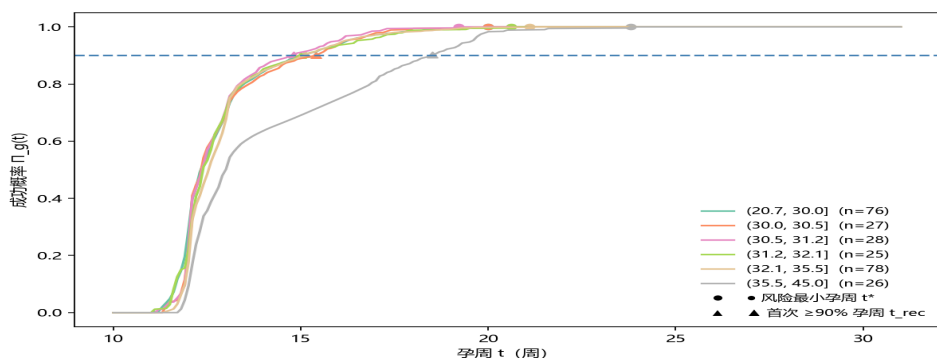


图 13: 不同 BMI 组的首检成功率曲线图

结合图中的实证结果可以清晰看到：低 BMI 组的曲线普遍陡峭，黑色圆点（风险最优时点）与黑色三角（首检成功率 90% 时点）几乎重合，说明误差对结果影响较弱。而在高 BMI 组中，曲线上升缓慢，圆点显著早于三角，表明检测误差显著改变了最佳检测时点的判断。

因此，检测误差不仅提高了总体风险水平，还系统性地推动最佳检测时点向更早孕周转移，尤其在高 BMI 人群中表现得更为突出。这一结果提示，在制定个性化的 NIPT 检测策略时，应当充分考虑检测误差的存在，不宜单纯依赖“达到 90% 成功率”的阈值，而应采用基于风险函数的综合指标，以确保推荐时点的稳健性和可行性。

九、 问题四的模型建立与求解

基于问题四的研究目标，本文结合 21、18、13 号染色体 Z 值、GC 含量、读段数及 BMI 等特征，构建了逻辑回归与 GBDT 融合的判别模型，并通过概率校准与阈值优化提升性能。为直观展示建模与求解思路，本文绘制了针对问题四的模型建立与求解流程图，如下所示。

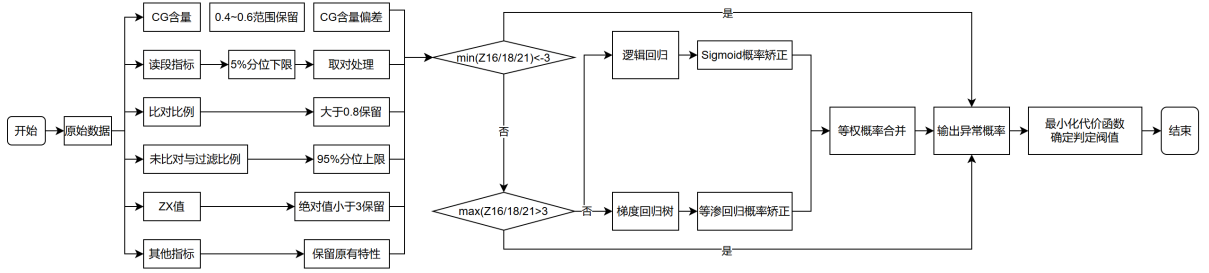


图 14: 问题四技术路线图

9.1 女胎异常自变量的选择

问题四旨在建议女胎异常的判断模型，对数据进行分析发现，数据集中并未存在胎儿非健康的现象。结合题意和上述情况，染色体倍数异常往往会导致婴儿健康异常的情况，因此本文将染色体异常的女胎视作为异常女胎进行判断模型的建立自变量：

$$y_{abnormal} = \begin{cases} 1, & n_{13}n_{18}n_{21} \neq 2 \\ 0, & \text{else} \end{cases} \quad (9.1)$$

其中，代指 13、18、21 号染色体的数量。

9.2 数据的筛选

在 NIPT 检测中，测序数据质量直接决定后续 Z 值与异常判定的可靠性。如果文库构建失败，比对率过低，GC 偏离严重，即使后续统计量计算正确，也可能因输入数据偏差导致假阳性或假阴性判别。因此在使用数据拟合女胎正、异常判别模型之前，对女胎一些重要指标数据进行筛选处理。

1) GC 含量

指标包括全基因组 GC 含量、13/18/21 染色体 GC 含量。人类基因组平均 $GC \approx 0.41$ ，偏离该范围说明测序片段选择或 PCR 偏差，可能导致某些染色体读段比例系统性增加。根据这一特性对 $GC \in [0.40, 0.60]$ 样本进行剔除。

同时 NIPT 领域里常采用 0.5 作为“中性基准”，因为这是最对称、也是测序最无偏的 GC 含量。因此对各 GC 量转化为：

$$|GC - 0.5|(GC \text{ 偏离度}) \quad (9.2)$$

2) 原始读段数与唯一比对读数

若原始读段数过低或噪声过高， Z 值波动过大。使用训练集的 5% 分位数作为下限，对低于下限数据剔除。同时取 $\log(1+x)$ 缩小量级影响，并使得分布更贴合正态。

3) 比对比例

高质量 NIPT 数据通常具有 85 – 95% 的成功比对比例 (A)，根据数据实际情况，令 $A \leq 0.80$ 数据进行剔除。

4) 未比对比例与被过滤比例

过高的未比对或过滤比例通常说明原始数据低质量或存在污染。使用 95% 分位数作为上限，可以剔除最差的 5% 样本。

5) Y 染色体 Z 值 (ZX)

若 ZX 显著偏离，可能是母体结构变异或文库偏差，结果不可靠。因此以 3 作为阈值进行处理，即剔除 $|ZX| > 3$ 数据。

6) 其他指标数据

年龄、 BMI 、孕期数据等指标利用第一问的数据预处理方式进行清洗。13/18/21 染色体的 Z 值 ($Z13, Z18, Z21$) 就是判断三体异常的核心信息。异常值的出现恰恰能体现女胎异常情况，因此不对其进行处理。

9.3 女胎正、异常判别模型的建立

9.3.1 初步判断机制

根据 NIPT 临床实践中， $Z \geq 3$ 是最常用的判定阳性标准。这个阈值对应于正态分布下 $p \approx 0.00135$ 的极端事件，几乎不可能由随机噪声产生，因此被认为是高置信度阴性。同理， $Z \leq -3$ 极可视为阴性样本。则有：

$$P_{abnormal} = \begin{cases} 99.5\%, & \min(Z13, Z18, Z21) \geq 3 \\ 0.5\%, & \max(Z13, Z18, Z21) \leq -3 \end{cases} \quad (9.3)$$

9.3.2 概率判断机制

1) 逻辑回归概率判断模型 逻辑回归基本形式为：

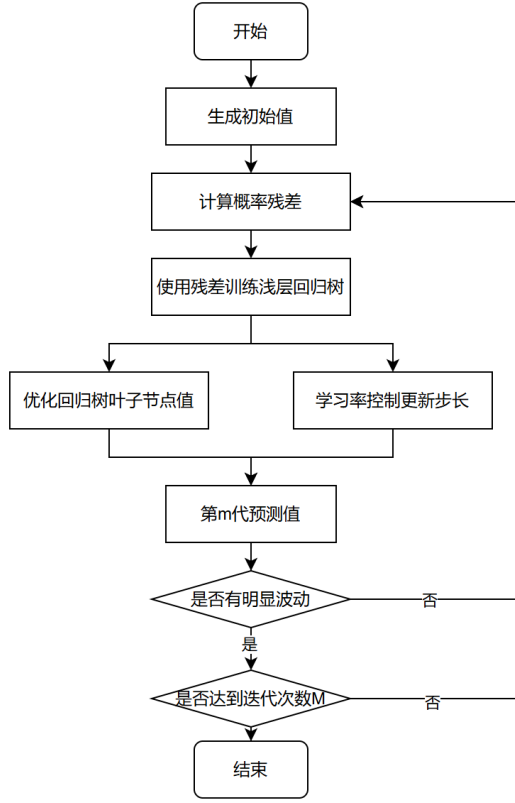
$$P_{LR} = \frac{1}{1 + \exp\left(-\left(\sum_{j=1}^p w_j x_j + b\right)\right)} \quad (9.4)$$

逻辑回归的参数 w_j, b 的获取通常使损失函数最小化获得：

$$\mathcal{L}(w, b) = -\frac{1}{N} \left[\sum_{i:y_i=1} \log P_{LR} + w_0 + \sum_{i:y_i=0} \log(1 - P_{LR}) \right] + \lambda \|w\|_2^2 \quad (9.5)$$

其中， w_1, w_0 表示阳阴变量的权重，根据样本比例计算，平衡阳性样本少、阴性样本多的情况； $\|w\|_2^2$ 表示所有参数的平方，采用 L2 正则化方式，防止过拟合的现象发生。

2) 梯度提升树概率判断模型



为描述模型迭代优化步骤及更准确刻画模型的训练与更新过程，本文采用梯度提升树的迭代公式展开推导与系统说明。

Step 1 对逻辑分类问题初始化，以常数函数 F_0 为起点，由训练集阳性率 $\bar{p}$ 作为初始概率。

Step 2 对每个样本 x_i ，在第 m 轮以前模型 $f_{m-1}(x_i)$ 输出推导得到概率 p_i ，数据损失的梯度： $r_{im} = y_i - p_i$ 。

Step 3 以为 (x_i, r_{im}) 训练数据，训练一棵浅层回归树 $h_m(x)$ 。

Step 4 对树的每个叶节点 R_{jm} 求最优系数以最小化该叶内的损失 γ_{jm} 。

Step 5 用学习率 $\nu \in (0, 1]$ 控制更新步长。重复 Step1-4，直至达到基树数上限 M ，或在验证集上性能不再上升。

Step 6 代入待测孕妇指标数据，可以得到模型分数 $F_{GBDT}(x)$ ，通过 Sigmoid 公式变换得到概率。

图 15: GBDT 迭代流程图

3) 逻辑回归和梯度提升树并列概率判断模型

通过将上述两种模型进行结合，兼具逻辑回归的可解释性与稳定性，以及梯度提升树^[5]的非线性表达与交互捕捉能力，从而在样本中的关键指标数据能被更好地读取，提升了判定准确性与鲁棒性。

3) 概率矫正：逻辑回归在样本不平衡或正则化作用下往往会导致概率整体偏高或偏低，而梯度提升树则容易输出过度自信的极端概率，因此需要采用不同方式对概率进行矫正。

对于逻辑回归，采用 Sigmoid 二参数校准：

$$P_{LR-cor}(x) = \frac{1}{1 + e^{Ap_{LR} + B}} \quad (9.6)$$

并通过最小化加权对数似然求得 A, B ：

$$\min_{A, B} \sum_i w_i [y_i \log \sigma(Af_i + B) + (1 - y_i) \log(1 - \sigma(Af_i + B))]. \quad (9.7)$$

对于梯度提升树，采用等渗回归校准函数。对所有概率样本和实际值的二元组合数据进行排序：

$$(\hat{p}(y_1), y(y_1)), (\hat{p}(y_2), y(y_2)), \dots, (\hat{p}(y_n), y(y_n)), \quad \text{其中 } \hat{p}(1) \leq \hat{p}(2) \leq \dots \leq \hat{p}(n).$$

通过最小化误差平方得到矫正函数 g 。

$$g = \arg \min \sum_{i=1}^n (g(\hat{p}(i)) - y(i))^2 \quad (9.8)$$

$$[0, 1] = \bigcup_{k=1}^K I_k, \quad g(\hat{p}) = \theta_k, \quad \forall \hat{p} \in I_k \quad (9.9)$$

最后可得矫正概率：

$$P_{GBDT-cor} = g(P_{GBDT}). \quad (9.10)$$

模型概率：逻辑回归和梯度提升树计算所得指标，经过概率矫正后，进行加权平均求得模型概率。为简化模型和考虑小样本下方差的鲁棒性，采用等权方式：

$$P_{abnormal} = \frac{1}{2}P_{LR-cor} + \frac{1}{2}P_{GBDT-cor}. \quad (9.11)$$

模型判定阈值的确定：为了找到模型最优的一个概率阈值来判定女胎是否异常，引用代价函数：

$$\min R(\tau) \quad (9.12)$$

其中，

$$R(\tau) = C_{FP}FP(\tau) + C_{FN}FN(\tau), \quad (9.13)$$

$FP(\tau)$ 表示阈值为 τ 时模型判定的假阳性人数， $FN(\tau)$ 表示阈值为 τ 时模型判定的假阴性人数； C_{FP}, C_{FN} 分别是假阳性和假阴性的代价权重。在医学筛查场景中，漏检比误报代价更高，常取 $C_{FN} \gg C_{FP}$ ，这里取 10 : 1。

9.4 模型求解：女胎异常的联合判定方法

9.4.1 交叉验证性能与阈值选择

1. 结果及分析为验证模型泛化能力，我们按建模部分所述流程：将逻辑回归与梯度提升树（GBDT）输出的异常概率做校准与加权平均，再用代价函数（FN:FP=10:1）在验证集上选取最优阈值 τ 。

在模型训练过程中，我们采用了五折交叉验证 + 留出集来评估不同模型在训练集上的稳健性，并选择最优的超参数与概率阈值。然而，交叉验证仍然在同一数据集内部进行拆分与评估，可能存在一定的数据泄漏风险。为了确保模型具有良好的泛化能力，我们在整体数据集中额外划分出一部分样本，构建留出集（Hold-out Set）。

留出集数据在整个建模与调参阶段均未被使用，因而能够真实反映模型在未知样本上的表现。最终我们将训练完成的模型直接应用于留出集，并计算混淆矩阵及多项分类指标（准确率、精确率、召回率、F1 值等），从而验证模型的实际有效性。交叉验证的关键指标与每折混淆矩阵统计如下图：

表 9: 混淆矩阵指标统计

fold	TP	FP	TN	FN	sensitivity	specificity
1	10	0	104	6	0.625	1.000
2	14	6	98	2	0.875	0.942
3	10	6	98	6	0.625	0.942
4	13	10	94	3	0.813	0.904
5	6	1	103	0	1.000	0.990

其中，TP 指被正确判定为阳性的样本数；FP 指被错误判定为阳性的阴性样本数；

TN 指被正确判定为阴性的样本数;FN 指被错误判定为阴性的阳性样本数；sensitivity（灵敏度）指阳性检出率；specificity（特异度）指阴性排除率。

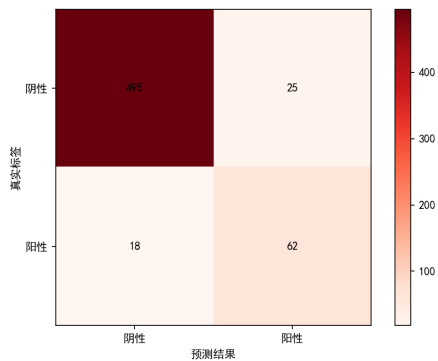
表 10: 交叉验证指标统计

fold	AUC	AP	brier	tau
1	0.7356	0.6913	0.0600	0.350
2	0.9087	0.8785	0.0368	0.177
3	0.8227	0.6800	0.0655	0.180
4	0.9213	0.8366	0.0382	0.157
5	0.9976	0.9838	0.0243	0.157

其中，AUC 指受试者工作特征曲线下面积，衡量模型区分阳性与阴性的能力；AP 指平均精确率（Average Precision），常用于样本不平衡场景下，评价模型在不同阈值下的综合检出性能；brier 指预测概率与真实标签的均方误差，反映概率输出的校准性；tau 指交叉验证中根据代价函数或性能最优化得到的最优判定阈值。

从结果来看，模型在交叉验证中整体表现稳健，AUC 均值接近 0.9，AP 保持较高水平，Brier 分数较低，表明判别力和概率校准性良好。混淆矩阵显示特异度始终大于 0.94，灵敏度在不同折次间波动（0.625-1.000），但整体达到了较好的检出能力与平衡性。因此，模型在女胎异常判定中具备较强的鲁棒性和应用潜力。

2. 测试集混淆矩阵（可视化）为进一步验证模型在留出集上的判别效果，绘制了测试集的混淆矩阵，如图所示：



该图展示了模型在留出集上的分类表现，其中真阴性（TN=495）、假阳性（FP=25）、假阴性（FN=18）、真阳性（TP=62）分布清晰。可以看出，模型在负样本上的判别较为准确，同时对正样本也保持了较高的检出能力，说明所选阈值在“漏报代价远高于误报”的设定下取得了较为合理的平衡。

图 16: 测试集混淆矩阵图

9.5 关键特征贡献结果

为保证可解释性，我们分别从 GBDT（基于分裂增益）与逻辑回归（基于系数绝对值）两个视角评估特征重要性。

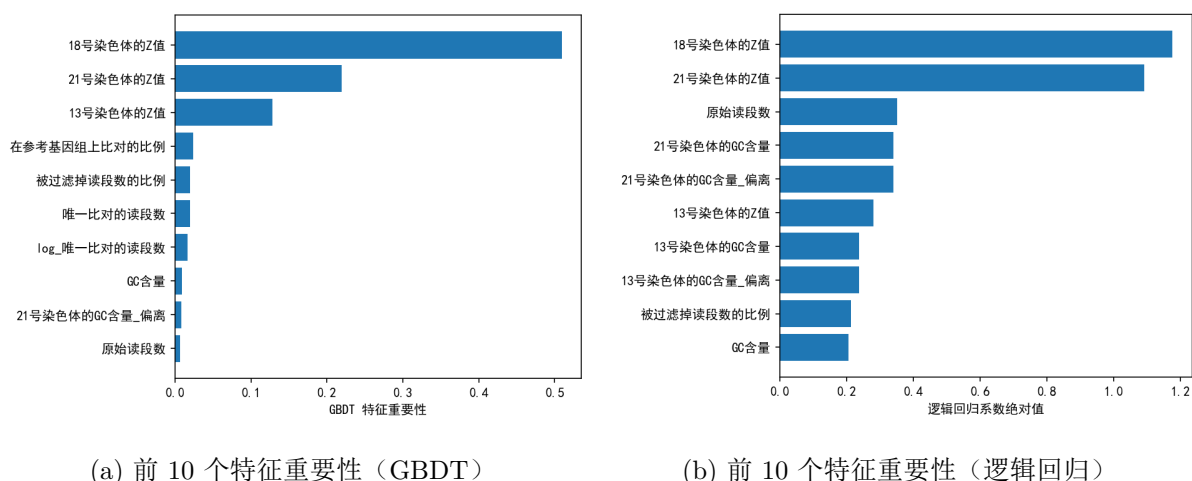


图 17: 女胎异常判定模型的特征重要性分析图

这两幅图分别展示了基于 **GBDT** 与 **逻辑回归** 的特征重要性排序，反映了模型在判定女胎异常时对不同输入变量的依赖程度。

- **GBDT 特征重要性图**：可以看出，18 号、21 号和 13 号染色体的 Z 值贡献最大，成为模型判断的核心依据，其次是部分读段比例和 GC 含量等指标。
- **逻辑回归特征重要性图**：结果同样突出 18 号和 21 号染色体 Z 值的重要性，同时 GC 含量及其偏离值、原始读段数等特征也具有较高权重，进一步支撑了染色体 Z 值作为关键判别指标的结论。

总结：两种方法尽管建模机制不同，但均表明 **13、18、21 号染色体 Z 值**是判定女胎异常的最核心特征，其他特征（如 GC 含量和读段相关指标）则起到辅助作用。这说明所建模型在特征选择上具有稳健性和合理性。

9.6 测试集性能评估结果

验证集 600 个样本中，有 520 个阴性样本，80 个阳性样本。为进一步验证模型在女胎异常判定任务中的性能表现，我们分别统计了验证集上的混淆矩阵指标（表 11）以及交叉验证的综合指标（表 12）。

表 11: 混淆矩阵指标统计

TP	FP	TN	FN	sensitivity	specificity
62	25	495	18	0.775	0.9519

表 12: 交叉验证指标统计

AUC	AP	brier	tau
0.8663	0.8156	0.0450	0.194

从表 11 可以看出，在 600 个样本中，模型正确识别阳性 62 例，漏判 18 例，阴性中正确识别 495 例，误判 25 例。由此计算得出灵敏度约为 0.775，特异度约为 0.952，说明模型在保持较高特异度的同时，仍具备较强的灵敏度。

表 12 显示了交叉验证下的整体性能： $AUC \approx 0.87$, $AP \approx 0.82$, $Brier \approx 0.045$, $t \approx 0.194$ ，各项指标与留出集结果一致，表明模型不仅在训练验证中表现稳健，而且在独立数据上也具备良好的泛化能力。整体上，该模型在“尽量避免漏报”的目标下实现了较为理想的平衡。结果说明模型不仅在交叉验证中表现稳定，在独立测试集上的泛化能力同样良好，能够较好地完成对女胎异常的判定任务。

9.7 问题四模型判定规则与任务完成度总结

本问最终建立了基于逻辑回归与 GBDT 的联合判定规则，并通过双重概率校准与加权平均获得异常概率。模型的性能与任务完成度总结如下：

1. **判定规则：**对同一样本，分别由逻辑回归与 GBDT 输出异常概率，经校准与加权平均后得到综合判定值。当异常概率 $P_{abnormal} \geq \tau$ 时判为“异常”，否则判为“阴性”。在代价敏感阈值 $\tau \approx 0.20$ (FN:FP = 10:1) 的条件下，该规则有效平衡了漏报与误报风险。

2. **性能表现：**交叉验证与留出集结果高度一致，表现为 AUC $\approx$ 0.88、AP $\approx$ 0.81、Brier $\approx$ 0.045，灵敏度 $\approx$ 0.78，特异度 $\approx$ 0.95，表明模型能够在尽量避免漏报的目标下保持较高的判别性能。

3. **可解释性：**特征重要性分析结果显示，21/18/13 号染色体的 Z 值是判定异常的核心依据，GC 含量、读段及比对指标则作为质量控制与辅助判别特征。混淆矩阵结果进一步证明所选阈值与代价函数设定匹配良好，保证了模型的稳健性与可用性。

综上所述，该结果不仅满足了题目要求的女胎异常判定任务，同时也体现了模型在准确性、鲁棒性与可解释性上的全面性。

十、 模型的评价

10.1 模型的优点

本文提出的“LR+GBDT+ 概率校准 + 规则层”复合式模型，相较于常见的简化方案（见表 13），在性能与鲁棒性方面展现出明显优势，具体体现在以下三点：

1. 性能提升：在保持较高灵敏度（0.775）的同时，将特异度提升至 0.952，仅 25 个假阳性，显著优于 LR-Simple（假阳性 92 个）和 GBDT-Simple（假阳性 56 个）。

2. 结果稳定：概率校准与规则层有效降低了阈值偏差，使 AP 与 Brier 分数达到最优（AP=0.816，Brier=0.045），保证输出的可靠性。

3. 应用价值：引入代价敏感阈值，兼顾漏报代价远高于误报的临床实际需求，实现“尽量避免漏报”的目标，增强了模型的可解释性与实用性。

表 13: 不同模型的性能比较

模型	AUC	AP	Brier	灵敏度	特异度
LR-Simple	0.809	0.579	0.161	0.725	0.823
GBDT-Simple	0.900	0.811	0.048	0.813	0.892
本文方法	0.866	0.816	0.045	0.775	0.952

10.2 模型的缺点

原始分位数法在高 BMI 小样本组存在不稳定性，可能因极端个体出现“高 BMI 组推荐更早”的反直觉结果。同时存在幸存者偏倚（只保留了 Y 染色体超过 4% 的数据），使推荐时点被系统性低估。

10.3 模型的改进方向

可以通过进一步对未达者进行右删失处理（KM 分位数）并引入单调约束（PAVA），使推荐时点随 BMI 单调非减，结论与生理机制一致，且在多种分箱与风险设定下保持稳健。此改进方法为后续推广应用奠定更加坚实的基础

参考文献

- [1] 施炜慧, 徐晨明. 无创产前检测在产科母体并发症和合并症诊断中的应用价值 [J]. 实用妇产科杂志, 2025,41 (08).
- [2] 张亮亮, 卓召振, 黄盛文, 等. 贵州省多中心 16798 例 NIPT-plus 结果回顾性分析 [J]. 贵州医药, 2025,49 (08).
- [3] 王佳燕, 陈敏, 吴莉, 等. 无创产前基因检测双胎 21、18 和 13 - 三体综合征的应用研究 [J]. 中国实用妇科与产科杂志, 2017,33 (05).
- [4] 陈建宝, 丁军军. 分位数回归技术综述 [J]. 统计与信息论坛, 2008 (03).
- [5] 蔡文学, 罗永豪, 张冠湘, 等. 基于 GBDT 与 Logistic 回归融合的个人信贷风险评估模型及实证分析 [J]. 管理现代化, 2017,37 (02).
- [6] ChatGPT, GPT-5, OpenAI, 2025-09-05~2025-09-07.
- [7] DeepSeek, DeepSeek-R1-0528, 深度求索 (DeepSeek) , 2025-09-05)

附录 A 人工智能使用情况说明

1. 工具与版本

工具	版本	公司	使用日期
ChatGPT	GPT-5	OpenAI	2025-09-05
DeepSeek	DeepSeek-R1-0528	深度求索	2025-09-05

2. 使用目的与环节

工具仅用于代码与表述的辅助生成，包括语言的润色，方案的可行性探讨，所有步骤经本队独立复核与改写，所有关键决策由参赛队独立完成。

3. 关键交互摘要

ChatGPT 5 Instant

请简要分析下Y 染色体浓度与孕妇的孕周数有什么关系

🔗 共享 ...

Y 染色体浓度与孕妇孕周数之间的关系主要体现在 **母体血浆游离胎儿 DNA (cffDNA)** 的检测中。简要分析如下：

1. 胎儿 DNA 的来源

- 胎儿 DNA 主要来自胎盘滋养层细胞。
- 随着孕周的增加，胎盘逐渐发育，释放到母体血浆中的胎儿 DNA 含量也会逐渐升高。

2. Y 染色体作为检测标志

- 在母体血液中，若检测到 Y 染色体片段，通常意味着胎儿为男孩。
- Y 染色体浓度的多少可作为 **胎儿 DNA 总量** 的间接指标。

3. 与孕周的关系

- 早孕期 (7-10周)**：胎儿 DNA 刚刚进入母体血浆，浓度较低，但一般已可检测到 Y 染色体。
- 中孕期 (11-20周)**：Y 染色体浓度随孕周增加而上升，检测灵敏度和稳定性增强。
- 晚孕期 (20周以后)**：胎儿 DNA 浓度继续增加，但增幅趋缓，浓度达到较高水平。

4. 临床意义

- 产前无创检测 (NIPT)**：利用孕妇血液中的 Y 染色体 DNA，可在孕早期判断胎儿性别和检测染色体异常。
- 动态变化规律**：总体趋势是 **随孕周增加，Y 染色体浓度逐渐升高**，但个体差异、胎盘状态等因素也会影响结果。

要不要我帮你画一个 **孕周数与 Y 染色体浓度的关系曲线图**，直观展示它们的变化趋势？

🗑️ 🖨️ 🔍 🔄 ⌂ ...

+ 问问任何问题

🔊 🔇

ChatGPT 也可能犯错。请核查重要信息。

ChatGPT 也可能犯错。请核查重要信息。

请你给出PCA降维的PYTHON代码



以下是使用Python实现PCA降维的代码，包含详细注释和示例：

```
python
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# 示例数据：假设我们有4个样本，每个样本有3个特征
X = np.array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9],
               [10, 11, 12]])

print("原始数据形状:", X.shape)
print("原始数据:\n", X)

# 步骤1：数据标准化（可选但推荐）
# PCA对变量的尺度敏感，通常需要先进行标准化
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("\n标准化后的数据:\n", X_scaled)

# 步骤2：使用sklearn的PCA进行降维
# 降到2维
pca = PCA(n_components=2)

# 拟合数据并转换
X_pca = pca.fit_transform(X_scaled)

print("\n降维后的数据形状:", X_pca.shape)
```

给 DeepSeek 发送消息

深度思考 联网搜索

内容由 AI 生成，请仔细甄别

4. 采纳和人工修改情况

对于合理的建议，我们经人工校核/重构/对比实验验证；对不合理模型思路/表述/思路建议不予采用。

附录 B 支撑材料文件列表

文件名	类型	简介
data_clean.py	PYTHON 代码文件	数据预处理
description.py	PYTHON 代码文件	描述性统计
q1.py	PYTHON 代码文件	主成分+LMM 模型
q2.v1.py	PYTHON 代码文件	两阶段产前检测风险优化模型
q2_v2.py	PYTHON 代码文件	检测误差处理对比模型
q3.py	PYTHON 代码文件	基于个体特征的产前检测优化模型
q4.py	PYTHON 代码文件	规则优先的混合型二分类诊断模型
robustness_analysis.py	PYTHON 代码文件	诊断模型鲁棒性分析
visualize_q1.py	PYTHON 代码文件	数据可视化

visualize_q4.py	PYTHON 代码文件	数据可视化
模型指标展示.xlsx	数据预处理文件	数据预处理结果
标准化数据.xlsx	数据预处理文件	选取指标标准化结果

附录 C 模型代码展示

附录三 q1.py

介绍：主成分+LMM 模型

```
import os, json, re, warnings
import numpy as np
import pandas as pd
import statsmodels.api as sm
from sklearn.decomposition import PCA

warnings.filterwarnings("ignore")
OBFUSCATED_CONSTELLATION_BUNKER = os.path.join("results",
"model_artifacts")
LACED_PINEAPPLE_LEDGER = os.path.join("results", "data_clean.csv")
os.makedirs(OBFUSCATED_CONSTELLATION_BUNKER, exist_ok=True)

# 标准化
def atropos_zephyr_normalizer(chrysalis):
    chrysalis = pd.to_numeric(chrysalis, errors="coerce")
    return (chrysalis - chrysalis.mean()) / chrysalis.std(ddof=0)

# 对指定列进行主成分分析(PCA)并返回降维后的主成分得分
def mandolin_topography_mill(diorama, petunia, cumvar=0.80):
    nebula_matrix = diorama[petunia].copy().dropna()
    if nebula_matrix.empty:
        return pd.DataFrame(index=diorama.index), {
            "explained_var_ratio": [], "cum_ratio": [], "n_keep": 0, "cols": petunia
        }
    zircon = (nebula_matrix - nebula_matrix.mean(0)) / nebula_matrix.std(0, ddof=0)
    panini = PCA().fit(zircon.values)
    ephemera = panini.explained_variance_ratio_
    scrolls = zircon.values @ panini.components_.T
    cumulus = np.cumsum(ephemera)
    orion = int(np.searchsorted(cumulus, cumvar) + 1)
    vellum = pd.DataFrame(
        scrolls[:, :orion],
        columns=[f"PC{k}" for k in range(1, orion + 1)],
        index=zircon.index
    )
    return vellum, {
```

```

        "explained_var_ratio": ephemera.tolist(),
        "cum_ratio": cumulus.tolist(),
        "n_keep": int(orion),
        "cols": petunia
    }

# 读取清洗表
if not os.path.exists(LACED_PINEAPPLE_LEDGER):
    raise FileNotFoundError(f'未找到清洗表： {LACED_PINEAPPLE_LEDGER}，请
先运行预处理脚本。")

obsidian_ledger = pd.read_csv(LACED_PINEAPPLE_LEDGER).rename(columns=lambda
x: str(x).strip())

# 读取孕周
if "检测孕周_小数周" in obsidian_ledger.columns:
    obsidian_ledger["孕周周"] = pd.to_numeric(obsidian_ledger["检测孕周_小数周"],
errors="coerce")
elif "孕周周" in obsidian_ledger.columns:
    obsidian_ledger["孕周周"] = pd.to_numeric(obsidian_ledger["孕周周"],
errors="coerce")
else:
    raise KeyError("清洗表缺少 孕周： 需要 '检测孕周_小数周' 或 '孕周周'。")

# 读取BMI和年龄
obsidian_ledger["BMI"] = pd.to_numeric(obsidian_ledger.get("孕妇BMI",
obsidian_ledger.get("BMI")), errors="coerce")
obsidian_ledger["年龄"] = pd.to_numeric(obsidian_ledger.get("年龄"), errors="coerce")

# 读取季节
if "季节" in obsidian_ledger.columns:
    obsidian_ledger["季节"] = pd.Categorical(obsidian_ledger["季节"], categories=["春",
夏", "秋", "冬"], ordered=True)
elif "季节_扩展" in obsidian_ledger.columns:
    obsidian_ledger["季节"] = pd.Categorical(
        obsidian_ledger["季节_扩展"].astype(str).str.replace("季", "", regex=False),
        categories=["春", "夏", "秋", "冬"], ordered=True
    )
else:
    raise KeyError("清洗表缺少 季节 列： 需要 '季节' 或 '季节_扩展'。")

# 对目标变量进行logit转换
if "Y浓度_logit" in obsidian_ledger.columns:
    obsidian_ledger["logitY"] = pd.to_numeric(obsidian_ledger["Y浓度_logit"],
errors="coerce")
elif "Y浓度_比例" in obsidian_ledger.columns:
    y_chimera = pd.to_numeric(obsidian_ledger["Y浓度_比例"], errors="coerce")

```

```

y_chimera = np.clip(y_chimera, 1e-6, 1-1e-6)
obsidian_ledger["logitY"] = np.log(y_chimera/(1-y_chimera))
else:
    raise KeyError("清洗表缺少 Y 浓度列：需要 'Y浓度_logit' 或 'Y浓度_比例'。")

# 技术/质量变量命名对齐
def _cryptic_numeric_adapter(col):
    return pd.to_numeric(obsidian_ledger[col], errors="coerce") if col in
obsidian_ledger.columns else np.nan

if "唯一比对读段数_log" not in obsidian_ledger.columns and "唯一比对的读段数" in
obsidian_ledger.columns:
    obsidian_ledger["唯一比对读段数_log"] = np.log(_cryptic_numeric_adapter("唯一比
对的读段数").replace(0, np.nan))
if "原始读段数_log" not in obsidian_ledger.columns and "原始读段数" in
obsidian_ledger.columns:
    obsidian_ledger["原始读段数_log"] = np.log(_cryptic_numeric_adapter("原始读段数
").replace(0, np.nan))
for labyrinth_source, labyrinth_target in {
    "在参考基因组上比对的比例":"比对比例",
    "重复读段的比例":"重复比例",
    "被过滤掉读段数的比例":"过滤比例",
    "GC含量":"GC",
}.items():
    if labyrinth_target not in obsidian_ledger.columns and labyrinth_source in
obsidian_ledger.columns:
        obsidian_ledger[labyrinth_target] = _cryptic_numeric_adapter(labyrinth_source)

# ID
obsidian_ledger["ID"] = (obsidian_ledger["孕妇代码"].astype(str) if "孕妇代码" in
obsidian_ledger.columns else obsidian_ledger.index.astype(str))

# 建模底表
necessary_bedrock = ["logitY", "孕周周", "BMI", "ID"]
mezzanine_table = obsidian_ledger.dropna(subset=[c for c in necessary_bedrock if c in
obsidian_ledger.columns]).copy()

# 标准化
chrysanthemum_continuum = ["孕周周", "BMI", "年龄",
                            "唯一比对读段数_log", "原始读段数_log", "比对比例", "
重复比例", "过滤比例", "GC"]
for chrysanthemum_col in chrysanthemum_continuum:
    if chrysanthemum_col in mezzanine_table.columns:
        mezzanine_table[chrysanthemum_col + "_z"] =
atropos_zephyr_normalizer(mezzanine_table[chrysanthemum_col])

# -----
# LMM 拟合 + PCA

```

```

# -----
gossamer_formula_zero = "logitY ~ bs(孕周周_z, df=4) + BMI_z"
albatross_fit_m0 = sm.MixedLM.from_formula(gossamer_formula_zero, groups="ID",
re_formula="1", data=mezzanine_table).fit(reml=False)
albatross_fit_m1 = sm.MixedLM.from_formula(gossamer_formula_zero, groups="ID",
re_formula="1+孕周周_z", data=mezzanine_table).fit(reml=False)

kaleidoscope_axes = [c for c in ["唯一比对读段数_log", "原始读段数_log", "比对比例", "
重复比例", "过滤比例", "GC"] if c in mezzanine_table.columns]
seraph_scores_frame, oubliette_manifest = mandolin_topography_mill(mezzanine_table,
kaleidoscope_axes, cumvar=0.80)
for col_cherub in seraph_scores_frame.columns:
    mezzanine_table[col_cherub] = seraph_scores_frame[col_cherub]
filigree_components = list(seraph_scores_frame.columns)

labyrinth_rhs = "bs(孕周周_z, df=4) + BMI_z + 年龄_z + C(季节, Treatment(reference='
春'))"
if filigree_components:
    labyrinth_rhs += " + " + " + ".join(filigree_components)
formula2_pca = f"logitY ~ {labyrinth_rhs}"

mariner_subset = mezzanine_table.dropna(subset=["logitY", "孕周周_z", "BMI_z", "ID", "季
节"] + filigree_components)
albatross_fit_pca = sm.MixedLM.from_formula(formula2_pca, groups="ID",
re_formula="1+孕周周_z",
data=mariner_subset).fit(reml=False)
albatross_fit_sub = sm.MixedLM.from_formula(gossamer_formula_zero, groups="ID",
re_formula="1+孕周周_z",
data=mariner_subset).fit(reml=False)

# 导出工件（供可视化）
# 1) 模型关键信息
(open(os.path.join(OBFUSCATED_CONSTELLATION_BUNKER, "summary.txt"), "w",
encoding="utf-8")
).write(str(albatross_fit_pca.summary()))

pd.Series({
    "N_subset": mariner_subset.shape[0],
    "AIC_M1_sub": albatross_fit_sub.aic,
    "AIC_M2_PCA": albatross_fit_pca.aic,
    "LR_stat": 2*(albatross_fit_pca.llf - albatross_fit_sub.llf),
    "df_diff": len(albatross_fit_pca.fe_params) - len(albatross_fit_sub.fe_params),
    "Scale_resid": albatross_fit_pca.scale
}).to_csv(os.path.join(OBFUSCATED_CONSTELLATION_BUNKER, "fit_metrics.csv"),
encoding="utf-8-sig")

```

```

# 2) 固定效应参数 & 协方差
albatross_fit_pca.fe_params.to_csv(os.path.join(OBFUSCATED_CONSTELLATION_BUNKER, "fe_params.csv"), header=["coef"], encoding="utf-8-sig")
albatross_fit_pca.cov_params().to_csv(os.path.join(OBFUSCATED_CONSTELLATION_BUNKER, "fe_cov.csv"), encoding="utf-8-sig")

# 3) 随机效应（毛毛虫图）
pd.DataFrame(albatross_fit_pca.random_effects).T.to_csv(os.path.join(OBFUSCATED_CONSTELLATION_BUNKER, "random_effects.csv"),
                                                         encoding="utf-8-sig")

# 4) 拟合值与残差
pd.DataFrame({"fitted": albatross_fit_pca.fittedvalues, "resid": albatross_fit_pca.resid}
             ).to_csv(os.path.join(OBFUSCATED_CONSTELLATION_BUNKER, "fitted_resid.csv"), encoding="utf-8-sig", index=False)

# 5) PCA 信息 + 建模底表（可视化）
json.dump(oubliette_manifest,
open(os.path.join(OBFUSCATED_CONSTELLATION_BUNKER, "pca_info.json"), "w",
encoding="utf-8"), ensure_ascii=False)
heliotrope_plot_columns = ["logitY", "孕周周", "BMI", "年龄", "季节", "ID"] +
kaleidoscope_axes + filigree_components
(mezzanine_table[heliotrope_plot_columns].copy()
 ).to_csv(os.path.join(OBFUSCATED_CONSTELLATION_BUNKER, "df0_for_plot.csv"), encoding="utf-8-sig", index=False)

# 6) 保存标准化后的指标
palimpsest_z_columns = [c for c in mezzanine_table.columns if c.endswith("_z")]
palimpsest_export = mezzanine_table[["ID"] + palimpsest_z_columns].copy()
palimpsest_export.to_csv(os.path.join(OBFUSCATED_CONSTELLATION_BUNKER, "standardized_vars.csv"),
                        encoding="utf-8-sig", index=False)

print("\n[完成] 主程序已输出建模工件到：",
os.path.abspath(OBFUSCATED_CONSTELLATION_BUNKER))

```

附录五 q2_v2.py

介绍：检测误差处理对比模型

```
import os
import re
import math
import warnings
from dataclasses import dataclass
from typing import Optional, Dict, Tuple, List

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import font_manager, rcParams

warnings.filterwarnings("ignore", category=FutureWarning)

# 参数设置
OBSIDIAN_LEDGER = r"附件.xlsx"
CARAVAN_OUTPOST = r"q2_outputs_compare"

KELP_THRESHOLD = 0.04
WOBBLE_BUCKETS = 10
JASMINE_FUSE_TOL = 2
ORBIT_ALPHA_MAIN = 0.80
ORBIT_ALPHA_SAFE = 0.90
DRAW_TRACES = True
GOLIATH_BOOTSTRAPS = 300
CRICKET_SEED = 42

# 时间代价与两阶段参数
HEPTAD = 7
COBALT_STAGE13_BASE = 2.0
LAMBDA_PER_HEPTAD = 0.20
STAGE27_CEILING = 3.4
STAGE12_WEIGHT = 1.0
STAGE13_27_WEIGHT = 2.0
STAGE28_WEIGHT = 5.0
UNIT_TEST_COST = 1.0
FIRST_MISS_PENALTY = 0.5
GAP_DELTA_DAYS = 21
# DELTA_NOISE 由各模式决定 (baseline=0; 误差A/误差B会覆盖)
TAU_GRID = np.arange(0.70, 0.96, 0.05)

np.random.seed(CRICKET_SEED)

# 中文字体
def prime_hanzi_fonts():
    candidates = [
        "Microsoft YaHei", "Microsoft YaHei UI", "SimHei", "SimSun",
        "PingFang SC", "Hiragino Sans GB", "Heiti TC",
```

```

        "Noto Sans CJK SC", "Noto Serif CJK SC", "WenQuanYi Micro Hei"
    ]
    installed = set(f.name.lower() for f in font_manager.fontManager.ttflist)
    chosen = None
    for name in candidates:
        if name.lower() in installed:
            chosen = name
            break
    if chosen is None:
        print("警告： 未检测到常见中文字体，建议安装微软雅黑/思源黑体。")
        chosen = "sans-serif"
    rcParams["font.sans-serif"] = [chosen]
    rcParams["font.family"] = "sans-serif"
    rcParams["axes.unicode_minus"] = False
    print(f'已启用中文字体： {chosen}')

# 工具函数
def ensure_burrow(path: str):
    if not os.path.exists(path):
        os.makedirs(path, exist_ok=True)

def sniff_columns(df: pd.DataFrame) -> Dict[str, str]:
    mapping = {"id": None, "ga": None, "bmi": None, "yfrac": None, "sex": None,
"sample_time": None}
    cols = list(df.columns)
    def pick(cands: List[str]) -> Optional[str]:
        for c in cols:
            cn = str(c).lower().replace(" ", "")
            for k in cands:
                if k in cn:
                    return c
        return None
    mapping["id"] = pick(["孕妇代码", "样本编号", "受检者", "病人", "编号", "patient", "id"])
    mapping["ga"] = pick(["检测孕周", "孕周", "gestational", "ga", "pregweek"])
    mapping["bmi"] = pick(["bmi"])
    mapping["yfrac"] = pick(["y染色体浓度", "y浓度", "y-浓度", "ychrom", "yfraction", "y_ratio", "yratio"])
    mapping["sex"] = pick(["性别", "胎儿性别", "fetal_sex", "sex"])
    mapping["sample_time"] = pick(["采样时间", "采血时间", "抽血时间", "检测时间", "sampletime", "collection", "date"])
    if mapping["id"] is None:
        df["__ROW_ID__"] = np.arange(1, len(df)+1)
        mapping["id"] = "__ROW_ID__"
    if mapping["ga"] is None:
        raise ValueError("未找到‘检测孕周’列。")
    if mapping["bmi"] is None:
        raise ValueError("未找到‘BMI’列。")
    if mapping["yfrac"] is None:

```

```

        raise ValueError("未找到‘Y染色体浓度’列。")
    return mapping

def parse_weeks_days(s: str) -> Optional[int]:
    if pd.isna(s): return None
    txt = str(s).strip().lower().replace("周", "w").replace("天", "d")
    m = re.match(r'^\s*(\d+)\s*w(?:\s*(\d+)\s*d)?$', txt)
    if m:
        w = int(m.group(1)); d = int(m.group(2)) if m.group(2) else 0
        return int(w*7 + d)
    m = re.match(r'^\s*(\d+)\s*\+\s*(\d+)\s*$', txt)
    if m:
        return int(m.group(1))*7 + int(m.group(2))
    m = re.match(r'^\s*(\d+)\s*$', txt)
    if m:
        return int(m.group(1))*7
    return None

def as_ww_plus_d(days: int) -> str:
    if pd.isna(days): return ""
    w = int(days)//7; d = int(days)%7
    return f"{w}w+{d}"

def to_y_fraction(x) -> Optional[float]:
    if pd.isna(x): return None
    try:
        v = float(x)
    except:
        try:
            s = str(x).strip().replace("%", "")
            v = float(s)
        except:
            return None
    if v < 0: return None
    if v > 1.0 and v <= 100.0: v = v/100.0
    return float(v)

def qtile(x: pd.Series, q: float) -> Optional[float]:
    x = pd.to_numeric(x, errors="coerce").dropna()
    if len(x) == 0: return None
    return float(np.quantile(x, q))

def filter_males(df: pd.DataFrame, id_col: str, y_col: str, sex_col: Optional[str]) -> pd.DataFrame:
    df = df.copy()
    if sex_col is not None:
        s = df[sex_col].astype(str).str.lower()
        male_mask = s.str.contains("男") | s.str.contains("male") | s.str.contains("boy")
        return df.loc[male_mask].copy()
    grp = df.groupby(id_col)[y_col].max().reset_index(name="ymax")

```



```

male_ids = set(grp.loc[grp["ymax"] >= 0.01, id_col])
return df.loc[df[id_col].isin(male_ids)].copy()

def first_hit_time(df: pd.DataFrame, id_col: str, ga_days_col: str, y_col: str, thr: float) ->
pd.DataFrame:
    records = []
    for pid, sub in df[[id_col, ga_days_col, y_col]].dropna().sort_values([id_col,
ga_days_col]).groupby(id_col):
        sub = sub.sort_values(ga_days_col)
        hit = sub.loc[sub[y_col] >= thr]
        if len(hit) > 0:
            t = int(hit.iloc[0][ga_days_col])
            records.append((pid, t, as_ww_plus_d(t), True))
        else:
            records.append((pid, np.nan, "", False))
    out = pd.DataFrame(records, columns=[id_col, "T_days", "T_wwd", "has_reached"])
    return out

def qbin(series: pd.Series, n_bins: int) -> pd.Categorical:
    n = n_bins
    while n >= 2:
        try:
            return pd.qcut(series, q=n, duplicates="drop")
        except Exception:
            n -= 1
    return pd.cut(series, bins=1)

@dataclass
class BucketStat:
    left: float
    right: float
    n: int
    p50: float
    p80: float
    p90: float

def summarize_buckets(df: pd.DataFrame, bin_col: str, t_col: str) -> List[BucketStat]:
    stats = []
    for cat, sub in df.groupby(bin_col, observed=True):
        bounds = [np.nan, np.nan]
        if hasattr(cat, "left"):
            bounds = [float(cat.left), float(cat.right)]
        t = pd.to_numeric(sub[t_col], errors="coerce").dropna()
        if len(t) == 0:
            stats.append(BucketStat(bounds[0], bounds[1], len(sub), np.nan, np.nan,
np.nan))
        else:
            stats.append(BucketStat(bounds[0], bounds[1], len(sub),
qtile(t, 0.5),
qtile(t, 0.8),

```

```

                                qtile(t, 0.9)))
stats = sorted(stats, key=lambda z: (z.left if not math.isnan(z.left) else -1e9))
return stats

def fuse_by_p80(stats: List[BucketStat], tol_days: int) -> List[BucketStat]:
    if len(stats) <= 1: return stats
    changed = True
    merged = stats
    while changed and len(merged) > 1:
        changed = False
        new_list = []
        i = 0
        while i < len(merged):
            if i < len(merged) - 1:
                a, b = merged[i], merged[i+1]
                if not (np.isnan(a.p80) or np.isnan(b.p80)) and abs(a.p80 - b.p80) <=
tol_days:
                    left = a.left; right = b.right; n = a.n + b.n
                    p50 = np.nanmean([a.p50, b.p50])
                    p80 = np.nanmean([a.p80, b.p80])
                    p90 = np.nanmean([a.p90, b.p90])
                    new_list.append(BucketStat(left, right, n, p50, p80, p90))
                    i += 2; changed = True; continue
            new_list.append(merged[i]); i += 1
        merged = new_list
    return merged

def replicate_sd(df: pd.DataFrame, id_col: str, ga_days_col: str, y_col: str) ->
pd.DataFrame:
    tmp = df.copy()
    tmp["GA_day_int"] = tmp[ga_days_col].round(0).astype("Int64")
    g = tmp.groupby([id_col,
"GA_day_int"])[y_col].agg(["count", "mean", "std"]).reset_index()
    g = g.rename(columns={"count": "n_repeats", "mean": "y_mean", "std": "y_sd"})
    g = g.loc[g["n_repeats"] >= 2].copy()
    return g

def slope_ols(x: np.ndarray, y: np.ndarray) -> Tuple[float, float, float]:
    x = np.asarray(x, dtype=float); y = np.asarray(y, dtype=float)
    X = np.c_[np.ones_like(x), x]
    beta = np.linalg.pinv(X).dot(y)
    yhat = X.dot(beta)
    ssr = np.sum((yhat - y.mean())**2)
    sse = np.sum((y - yhat)**2)
    r2 = ssr / (ssr + sse) if (ssr + sse) > 0 else 0.0
    return float(beta[1]), float(beta[0]), float(r2)

def bootstrap_quantiles_by(df: pd.DataFrame, group_col: str, t_col: str,
                           q_list=(0.8,0.9), B: int=300, seed: int=42) ->
pd.DataFrame:

```

```

rng = np.random.default_rng(seed)
rows = []; keys = sorted(df[group_col].unique(), key=lambda x: str(x))
for g in keys:
    sub = df.loc[(df[group_col]==g) & (df["has_reached"]==True)]
    t = sub[t_col].dropna().values
    if len(t) == 0:
        for q in q_list: rows.append((g,q,np.nan,np.nan,np.nan))
        continue
    for q in q_list:
        point = float(np.quantile(t, q))
        if B <= 0 or len(t) < 2:
            rows.append((g,q,point,np.nan,np.nan)); continue
        boots = []
        n = len(t)
        for _ in range(B):
            samp = rng.choice(t, size=n, replace=True)
            boots.append(np.quantile(samp, q))
        lo, hi = float(np.quantile(boots,0.025)), float(np.quantile(boots,0.975))
        rows.append((g,q,point,lo,hi))
return pd.DataFrame(rows, columns=[group_col,"quantile","point","lo","hi"])

# —— 时间代价曲线 ——
def time_cost_curve(day: float) -> float:
    if day <= 12 * HEPTAD:
        return STAGE12_WEIGHT
    if day < 27 * HEPTAD:
        weeks_since_13 = (day - 13 * HEPTAD) / HEPTAD
        val = COBALT_STAGE13_BASE + LAMBDA_PER_HEPTAD *
weeks_since_13
        val = min(val, STAGE27_CEILING, STAGE28_WEIGHT - 1e-6)
        return float(val)
    return STAGE28_WEIGHT
V_TIME_COST = np.vectorize(time_cost_curve)

# —— 原命中版（硬命中） ——
def eval_tau_hard(T: np.ndarray, tau: float,
                  delta_days: int, delta_noise: float) -> Dict[str, float]:
    q = float(np.quantile(T, tau))
    tau_day = q + delta_noise
    hit = (T <= tau_day)
    hit_rate = hit.mean()
    confirm = np.where(hit, tau_day, tau_day + delta_days).astype(float)
    tests = np.where(hit, 1, 2).astype(float)
    ind_penalty = np.where(hit, 0.0, FIRST_MISS_PENALTY).astype(float)
    R_stage = V_TIME_COST(confirm).mean()
    R_test = UNIT_TEST_COST * tests.mean()
    R_ind = ind_penalty.mean()
    R_total = R_stage + R_test + R_ind
    return {
        "tau": tau, "tau_day": tau_day,

```

```

        "hit_rate": float(hit_rate),
        "E_confirm_day": float(np.mean(confirm)),
        "E_tests": float(np.mean(tests)),
        "R_stage": float(R_stage),
        "R_test": float(R_test),
        "R_ind": float(R_ind),
        "R_total": float(R_total)
    }

# ===== 标准正态 CDF（用于方案B） =====
def phi(x: float) -> float:
    return 0.5 * (1.0 + math.erf(x / math.sqrt(2.0)))

# ===== 概率命中版（方案B） =====
def eval_tau_prob(T: np.ndarray, tau: float,
                  delta_days: int,
                  delta_noise: float,
                  sigma: float,
                  dVdt: float) -> Dict[str, float]:
    """ s_day = sigma / dVdt;  p_hit =  $\Phi((\tau_{\text{day}} - T)/s_{\text{day}})$  """
    q = float(np.quantile(T, tau))
    tau_day = q + delta_noise
    s_day = max(1e-6, sigma / max(1e-8, dVdt))
    z = (tau_day - T) / s_day
    p_hit = np.array([phi(float(v)) for v in z], dtype=float)

    hit_rate = float(p_hit.mean())
    E_confirm = (p_hit * tau_day + (1. - p_hit) * (tau_day + delta_days)).mean()
    E_tests = (p_hit * 1.0 + (1. - p_hit) * 2.0).mean()

    R_stage = float(V_TIME_COST(E_confirm))
    R_test = UNIT_TEST_COST * E_tests
    R_ind = FIRST_MISS_PENALTY * (1. - hit_rate)
    R_total = float(R_stage + R_test + R_ind)

    return {
        "tau": tau, "tau_day": tau_day, "hit_rate": hit_rate,
        "E_confirm_day": float(E_confirm), "E_tests": float(E_tests),
        "R_stage": float(R_stage), "R_test": float(R_test),
        "R_ind": float(R_ind), "R_total": float(R_total)
    }

# 从数据估计噪声参数
def estimate_noise_params(df_male, id_col: str, ga_col: str, y_col: str):
    # 1) 同日复测 SD 中位数
    within = replicate_sd(df_male, id_col=id_col, ga_days_col=ga_col, y_col=y_col)
    sigma_hat = float(np.nanmedian(within["y_sd"])) if len(within) > 0 else 0.005

    # 2) 稳健 dV/dt: 分箱-中位-中心差分

```

```

bin_w = 3
grid = np.arange(70, 141, 1)
med = []
for t in grid:
    win = df_male[(df_male[ga_col].between(t-bin_w, t+bin_w))]
    y = win[y_col].dropna()
    med.append(np.nanmedian(y) if len(y) >= 20 else np.nan)
med = pd.Series(med, index=grid).interpolate(limit_direction="both")

t0 = float(med.index[np.nanargmin(np.abs(med - 0.04))])
# 若边界不足, 回退到经验值
if t0-2 in med.index and t0+2 in med.index and np.isfinite(med.loc[t0-2]) and
np.isfinite(med.loc[t0+2]):
    dVdt = float((med.loc[t0+2] - med.loc[t0-2]) / 4.0) # /天
else:
    dVdt = 0.0013 / 7.0

# 3) 夹逼 (理性区间)
DV_FLOOR = 0.00012 # ≈0.084%/周
DV_CEIL = 0.00030 # ≈0.21%/周
dVdt = float(np.clip(dVdt, DV_FLOOR, DV_CEIL))

# 4) 计算 Δt 并上限 28 天
z90 = 1.2816
delta_noise = float(min((z90 * sigma_hat) / dVdt, 28.0))
return sigma_hat, dVdt, delta_noise

# 单模式运行封装
def run_single_mode(mode_name: str,
                    out_dir: str,
                    df_male: pd.DataFrame,
                    id_col: str, ga_col: str, bmi_col: str, y_col: str,
                    use_prob: bool,
                    delta_noise: float,
                    sigma_hat: float,
                    dVdt: float) -> Tuple[pd.DataFrame, pd.DataFrame,
pd.DataFrame, pd.DataFrame]:

    ensure_burrow(out_dir)

    # 1) 个体首次达标 T
    ttab = first_hit_time(df_male, id_col=id_col, ga_days_col=ga_col,
                          y_col=y_col, thr=KELP_THRESHOLD)
    pid_bmi = df_male.groupby(id_col)[bmi_col].median().reset_index()
    ttab = ttab.merge(pid_bmi, on=id_col, how="left")
    ttab_reached = ttab.loc[ttab["has_reached"] == True].copy()

    # 2) BMI 分箱与合并
    init_bins = qbin(ttab_reached[bmi_col], n_bins=WOBBLE_BUCKETS)

```

```

ttab_reached["BMI_bin_init"] = init_bins
init_stats = summarize_buckets(ttab_reached, "BMI_bin_init", "T_days")
merged_stats = fuse_by_p80(init_stats, tol_days=JASMINE_FUSE_TOL)
merged_labels = [(s.left, s.right) for s in merged_stats]
def label_merged(bmi):
    for (lft, rgt) in merged_labels:
        if bmi >= lft and bmi <= rgt:
            return f"[{lft:.1f}, {rgt:.1f})"
    return np.nan
ttab_reached["BMI_bin_merged"] = ttab_reached[bmi_col].apply(label_merged)

```

3) p50/p80/p90 基础表

```

rows_final = []
for g, sub in ttab_reached.groupby("BMI_bin_merged"):
    if pd.isna(g): continue
    t = sub["T_days"].dropna().values
    n = len(t)
    p50 = float(np.quantile(t, 0.5))
    p80 = float(np.quantile(t, 0.8))
    p90 = float(np.quantile(t, 0.9))
    rows_final.append([g, n, p50, p80, p90,
                        as_ww_plus_d(int(round(p50))),
                        as_ww_plus_d(int(round(p80))),
                        as_ww_plus_d(int(round(p90)))])
tab_final = pd.DataFrame(rows_final, columns=[
    "BMI区间", "样本数", "T_p50_days", "T_p80_days", "T_p90_days",
    "T_p50_ww+d", "T_p80_ww+d", "T_p90_ww+d"
]).sort_values("BMI区间")
tab_final["推荐首测(≈p80)"] = tab_final["T_p80_ww+d"]
tab_final["保守首测(≈p90)"] = tab_final["T_p90_ww+d"]

```

4) bootstrap 置信区间

```

ci_tbl = bootstrap_quantiles_by(ttab_reached, group_col="BMI_bin_merged",
t_col="T_days",
                                q_list=(0.8, 0.9),
                                B=GOLIATH_BOOTSTRAPS, seed=CRICKET_SEED)
def _mk_ci_row(q):
    sub = ci_tbl.loc[ci_tbl["quantile"] == q].copy()
    sub = sub.rename(columns={"point": f"p{int(q*100)}_days",
                              "lo": f"p{int(q*100)}_lo", "hi":
f"p{int(q*100)}_hi"})
    return sub[["BMI_bin_merged", f"p{int(q*100)}_days", f"p{int(q*100)}_lo",
f"p{int(q*100)}_hi"]]
ci80_tbl = _mk_ci_row(0.8); ci90_tbl = _mk_ci_row(0.9)
ci_merged = ci80_tbl.merge(ci90_tbl, on="BMI_bin_merged", how="outer")
tab_final = tab_final.merge(ci_merged, left_on="BMI区间",
right_on="BMI_bin_merged", how="left").drop(columns=["BMI_bin_merged"])

```

5) 两阶段风险-最优化 (逐组扫 τ)

```

rows_risk = []
curves_dict = {}
for g, sub in ttab_reached.groupby("BMI_bin_merged"):
    if pd.isna(g): continue
    T = sub["T_days"].dropna().values.astype(float)
    lines = []
    for tau in TAU_GRID:
        if use_prob:
            r = eval_tau_prob(T, tau,
                               delta_days=GAP_DELTA_DAYS,
                               delta_noise=delta_noise,
                               sigma=sigma_hat,
                               dVdt=dVdt)
        else:
            r = eval_tau_hard(T, tau,
                              delta_days=GAP_DELTA_DAYS,
                              delta_noise=delta_noise)

        r["BMI区间"] = g
        lines.append(r)
    rtab_df = pd.DataFrame(lines).sort_values("tau")
    curves_dict[g] = rtab_df.copy()

# 最优  $\tau^*$ 
best_row = rtab_df.sort_values("R_total").iloc[0].to_dict()
rows_risk.append(best_row)

# 单模式图
if DRAW_TRACES:
    base_dir = os.path.join(out_dir, "risk_by_group"); ensure_burrow(base_dir)
    def _save_fig(fig, name):
        fig.tight_layout()
        fig.savefig(os.path.join(base_dir,
                                  f"{name}_{g.replace('[', '').replace(']', '').replace(',', '_').replace(' ', '')}.png"), dpi=150)
        plt.close(fig)

    fig = plt.figure()
    plt.plot(rtab_df["tau"], rtab_df["R_total"])
    plt.xlabel("候选分位数  $\tau$ "); plt.ylabel("总风险（期望）")
    plt.title(f"[{mode_name}] 两阶段：总风险-分位数（{g}）")
    _save_fig(fig, "图_总风险_vs_分位数")

    fig = plt.figure()
    plt.plot(rtab_df["tau"], rtab_df["hit_rate"])
    plt.xlabel("候选分位数  $\tau$ "); plt.ylabel("首检命中率")
    plt.title(f"[{mode_name}] 两阶段：首检命中率-分位数（{g}）")
    _save_fig(fig, "图_命中率_vs_分位数")

risk_best = pd.DataFrame(rows_risk)
if not risk_best.empty:

```

```

        risk_best["tau_ww+d"] =
risk_best["tau_day"].round().astype(int).map(as_ww_plus_d)
        risk_best["E_confirm_ww+d"] =
risk_best["E_confirm_day"].round().astype(int).map(as_ww_plus_d)
        risk_best = risk_best[[
            "BMI区间
", "tau", "tau_day", "tau_ww+d", "hit_rate", "E_confirm_day", "E_confirm_ww+d", "E_tests",
            "R_stage", "R_test", "R_ind", "R_total"
        ]].sort_values("BMI区间")

# 保存
ensure_burrow(out_dir)
tab_final.to_csv(os.path.join(out_dir, "Q2_BMI_groups_recommendation.csv"),
index=False, encoding="utf-8-sig")
ttab.to_csv(os.path.join(out_dir, "Q2_subject_first_reach_time.csv"), index=False,
encoding="utf-8-sig")
risk_best.to_csv(os.path.join(out_dir, "Q2_risk_opt_best_tau.csv"), index=False,
encoding="utf-8-sig")

return tab_final, risk_best, ttab, curves_dict

# 主流程
def main():
    prime_hanzi_fonts()
    ensure_burrow(CARAVAN_OUTPOST)

# 读取与清洗
raw_frame = pd.read_excel(OBSIDIAN_LEDGER, engine="openpyxl")
mapping = sniff_columns(raw_frame)
col_id, col_ga, col_bmi, col_y, col_sex = mapping["id"], mapping["ga"],
mapping["bmi"], mapping["yfrac"], mapping["sex"]
frame = raw_frame.copy()
frame["GA_days"] = frame[col_ga].apply(parse_weeks_days)
frame["Y_frac"] = frame[col_y].apply(to_y_fraction)
frame["BMI_val"] = pd.to_numeric(frame[col_bmi], errors="coerce")
frame = frame.dropna(subset=[col_id, "GA_days", "BMI_val", "Y_frac"]).copy()
frame_male = filter_males(frame, id_col=col_id, y_col="Y_frac", sex_col=col_sex)

# 估计噪声参数
sigma_top, dVdt_est, delta_hat = estimate_noise_params(frame_male, id_col=col_id,
ga_col="GA_days", y_col="Y_frac")
print(f'[噪声参数] sigma_hat={sigma_top:.4f}（绝对比值），dV/dt={dVdt_est:.6f}
(/天)，Δt≈{delta_hat:.1f} 天")

# 三种模式
MODALITIES = [
    # 名称, use_prob, delta_noise
    ("no_error", False, 0.0), # baseline: 无误差
    ("error_A", False, float(delta_hat)), # 方案A: 顺延注入

```



```

        ("error_B", True, float(delta_hat)), # 方案B: 概率命中（同时也可叠加
        顺延）
    ]

    best_tabs_all = []
    curves_by_mode = {}
    p8090_tabs = []

    for name, use_prob, delta_noise in MODALITIES:
        out_dir = os.path.join(CARAVAN_OUTPOST, name)
        tab_final, best_tab, ttab, curves = run_single_mode(
            mode_name=name,
            out_dir=out_dir,
            df_male=frame_male,
            id_col=col_id, ga_col="GA_days", bmi_col="BMI_val", y_col="Y_frac",
            use_prob=use_prob,
            delta_noise=delta_noise,
            sigma_hat=sigma_top,
            dVdt=dVdt_est
        )
        best_tab["mode"] = name
        best_tabs_all.append(best_tab)
        curves_by_mode[name] = curves
        # 保存 p80/p90（原始 & 加顺延的 ww+d）
        tmp = tab_final[["BMI区间", "T_p80_days", "T_p90_days"]].copy()
        tmp["p80_ww+d"] =
tmp["T_p80_days"].round().astype(int).map(as_ww_plus_d)
        tmp["p90_ww+d"] =
tmp["T_p90_days"].round().astype(int).map(as_ww_plus_d)
        tmp["p80_ww+d_shifted"] = (tmp["T_p80_days"] +
delta_noise).round().astype(int).map(as_ww_plus_d)
        tmp["p90_ww+d_shifted"] = (tmp["T_p90_days"] +
delta_noise).round().astype(int).map(as_ww_plus_d)
        tmp["mode"] = name
        p8090_tabs.append(tmp)

    # 汇总最优  $\tau^*$  对比
    summary_dir = os.path.join(CARAVAN_OUTPOST, "summary");
    ensure_burrow(summary_dir)
    best_all_df = pd.concat(best_tabs_all, ignore_index=True)
    best_all_df = best_all_df[["mode", "BMI区间", "tau", "tau_day", "tau_ww+d",
"hit_rate", "E_confirm_day", "E_confirm_ww+d", "E_tests", "R_total"]].sort_values(["BMI区间", "mode"])
    best_all_df.to_csv(os.path.join(summary_dir, "best_tau_comparison.csv"),
index=False, encoding="utf-8-sig")

    # p80/p90 与“顺延修正”对比
    ptab_df = pd.concat(p8090_tabs, ignore_index=True)

```

```

ptab_df.to_csv(os.path.join(summary_dir, "p80_p90_with_shift.csv"), index=False,
encoding="utf-8-sig")

# 重叠风险曲线对比图（按 BMI 组各画一张，三模式叠加）
# 注意：不指定颜色
first_mode = next(iter(curves_by_mode.keys()))
group_keys = list(curves_by_mode[first_mode].keys())
for g in group_keys:
    fig = plt.figure()
    for name, _, _ in MODALITIES:
        if g in curves_by_mode[name]:
            rtab = curves_by_mode[name][g]
            plt.plot(rtab["tau"], rtab["R_total"], label=name)
    plt.xlabel("候选分位数  $\tau$ "); plt.ylabel("总风险（期望）")
    plt.title(f"三模式重叠：总风险-分位数（{g}）")
    plt.legend()
    fig.tight_layout()
    fig.savefig(os.path.join(summary_dir,
f'overlay_risk_{g.replace('[', '').replace(']', '').replace(',', '_').replace(' ', '')}.png"), dpi=150)
    plt.close(fig)

print("="*80)
print("【对比完成】请查看：")
print(f"- {summary_dir}/best_tau_comparison.csv ← 核心表（三模式最优  $\tau^*$  对比）")
print(f"- {summary_dir}/overlay_risk_*.png ← 重叠曲线图（峰值右移一目了然）")
print(f"- {summary_dir}/p80_p90_with_shift.csv ← p80/p90 及其顺延后建议时点")
print("="*80)

if __name__ == "__main__":
    main()

```

附录六 q3. py

介绍：基于个体特征的产前检测优化模型

```

import os, re, math, warnings
import numpy as np
import pandas as pd
from typing import List, Dict
from dataclasses import dataclass

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.isotonic import IsotonicRegression

import matplotlib.pyplot as plt
from matplotlib import font_manager, rcParams
warnings.filterwarnings("ignore", category=UserWarning)

# ===== 参数区 =====
QUASAR_LEDGER = "附件.xlsx"
AURORA_TABCODE = 0

PUMPKIN_TAU = 0.04          # 达标阈值 4%
FROST_ALPHA = 0.90          # 命中率约束

GRAIL_QUANTILES = [0.3,0.4,0.5,0.6,0.7,0.8,0.9]
TUNDRA_GRID = np.round(np.arange(10.0, 31.01, 0.1), 2) # 周

# —— 风险函数权重 ——
OPAL_WEIGHT_TIME = 0.25      # 纯“时间成本”权重（不乘1-Π）
OPAL_WEIGHT_DET = 0.75       # 检测次数的成本（期望次数 ~ 1 + (1-Π)）
OPAL_WEIGHT_FAIL = 1.00      # 首检失败/流程损耗
LARK_EARLY_FENCE = 12.0
LARK_LATE_FENCE = 28.0
TUSK_RISK_WEIGHTS = (1.0, 3.0, 6.0) # ≤12, 13–27, ≥28 的分段时间成本

# —— Q2 风格分组（等频细箱→p80≤2天相邻合并） ——
WOBBLE_BINS_INIT = 10
SAGE_P80_TOL = 2
DELTA_WEEKS_GLUE = 0.5 # 推荐点相邻合并的阈值

# 图表
QUILL_PLOTS_ONLY = True

np.random.seed(42)

# ===== 中文字体 =====
def mount_hanzi():
    cands = ["Microsoft YaHei", "SimHei", "PingFang SC", "Hiragino Sans GB",
             "Noto Sans CJK SC", "WenQuanYi Micro Hei"]
    installed = {f.name for f in font_manager.fontManager.ttflist}

```

```

chosen = None
for name in cands:
    if name in installed:
        chosen = name; break
if chosen is None:
    print("[字体] 未检测到常见中文字体，建议安装微软雅黑或思源黑体；暂用系统默认。")
    return
rcParams["font.sans-serif"] = [chosen]
rcParams["font.family"] = "sans-serif"
rcParams["axes.unicode_minus"] = False
print(f"[字体] 已启用中文字体：{chosen}")

# ===== 列名匹配 =====
DRIFT_ALIAS_MAP = {
    "id": ["孕妇代码", "受检编号", "样本号", "孕妇编号", "编号", "序号", "id"],
    "ga": ["检测孕周", "孕周", "gestational_age", "GA", "孕周(周)", "孕周周"],
    "y_pct": ["Y染色体浓度", "Y 染色体浓度", "Y浓度", "Y%", "Y_percent", "y%", "y_percent"],
    "bmi": ["孕妇BMI", "BMI", "体质指数"],
    "age": ["年龄", "母亲年龄", "孕妇年龄", "age"],
    "uniq": ["唯一比对的读段数", "唯一比对的读段数", "唯一比对读段数", "unique_mapped_reads", "uniq"],
    "raw": ["原始读段数", "raw_reads", "reads"],
    "mapr": ["在参考基因组上比对的比例", "比对比例", "mapping_rate", "map_ratio"],
    "dupr": ["重复读段的比例", "重复比例", "dup_ratio", "duplicate_rate"],
    "filr": ["被过滤掉读段数的比例", "过滤比例", "filter_ratio", "qc_filter_rate"],
    "gc": ["GC含量", "GC", "GC%", "gccontent"],
    "pc1": ["PC1", "pc1"], "pc2": ["PC2", "pc2"], "pc3": ["PC3", "pc3"],
    "pc4": ["PC4", "pc4"]
}
def sniff_alias(df, aliases: List[str]):
    cols = list(df.columns)
    norm = {c: re.sub(r"\s+", "", str(c)).lower() for c in cols}
    for a in aliases:
        key = re.sub(r"\s+", "", a).lower()
        for c, nc in norm.items():
            if key==nc or key in nc: return c
    return None
def harvest_columns(df)->Dict[str,str]:
    got={}
    for k,v in DRIFT_ALIAS_MAP.items():
        c = sniff_alias(df, v)
        if c: got[k]=c
    if "id" not in got:
        df["_SYN_ID_"] = np.arange(len(df))+1
        got["id"] = "_SYN_ID_"
    for k in ["ga", "y_pct", "bmi"]:

```

```

        if k not in got: raise ValueError(f'缺少关键列: {k}; 可用列
= {list(df.columns)}")
    print("[列名映射]")
    for k in sorted(got.keys()):
        print(f"    {k:>5s}    <-    {got[k]}")
    return got

# ===== 预处理 =====
def vapor_parse_ga(v)->float:
    if pd.isna(v): return np.nan
    s = str(v).strip().lower()
    if re.match(r"^\d+(\.\d+)?$", s): return float(s)
    s = s.replace("w", "周").replace("d", "天").replace("week", "周")
    m = re.match(r"^\d+)\s*\+?\s*(\d+)$", s)
    if m: return float(m.group(1)) + float(m.group(2))/7.0
    m = re.match(r"^\d+)\s*周\s*(\d+)\s*天?$", s)
    if m: return float(m.group(1)) + float(m.group(2))/7.0
    m = re.match(r"^\d+)\s*周$", s)
    if m: return float(m.group(1))
    nums = re.findall(r"\d+\.\d*", s)
    return float(nums[0]) if nums else np.nan
def morph_fraction(y):
    if pd.isna(y): return np.nan
    if isinstance(y, (int, float)): return float(y)/100.0 if y>1 else float(y)
    s = str(y).replace("%", "").strip()
    v = float(s); return v/100.0 if v>1 else v
def mint_attain_earliest(g: pd.DataFrame, tau=PUMPKIN_TAU)->float:
    sub = g.sort_values("GA")
    hit = sub.loc[sub["Y_frac"]>=tau, "GA"]
    return hit.iloc[0] if len(hit)>0 else np.nan

# ===== 质量 → PCA =====
def brew_quality_pca(df, got):
    if all(k in got for k in ["pc1", "pc2", "pc3", "pc4"]):
        pcs = df[[got["pc1"], got["pc2"], got["pc3"], got["pc4"]]].copy()
        pcs.columns = ["PC1", "PC2", "PC3", "PC4"]; return pcs, None
    q_cols = [got[k] for k in ["uniq", "raw", "mapr", "dupr", "filr", "gc"] if k in got]
    if len(q_cols)<3: raise ValueError("质量因子原始列过少, 无法做 PCA。")
    Q = df[q_cols].copy()
    for c in q_cols:
        low=str(c).lower()
        if ("读段" in c) or ("read" in low) or ("uniq" in low) or ("raw" in low):
            Q[c] = np.log1p(pd.to_numeric(Q[c], errors="coerce"))
        else:
            Q[c] = pd.to_numeric(Q[c], errors="coerce")
    Q = Q.replace([np.inf, -np.inf], np.nan).fillna(Q.median())
    Z = StandardScaler().fit_transform(Q.values)
    pca = PCA(n_components=4, random_state=42)
    PC = pca.fit_transform(Z)

```

```

pcs = pd.DataFrame(PC, columns=["PC1","PC2","PC3","PC4"], index=df.index)
return pcs, pca.explained_variance_ratio_

# ===== 分位数GBDT & 反演CDF =====
@dataclass
class QuartzQuantile:
    q: float
    model: GradientBoostingRegressor
def forge_quantile_engines(X, y, q_list):
    engines={}
    for q in q_list:
        m = GradientBoostingRegressor(
            loss="quantile", alpha=q,
            n_estimators=400, max_depth=3, min_samples_leaf=30,
            learning_rate=0.05, subsample=0.8, random_state=42
        )
        m.fit(X,y); engines[q]=QuartzQuantile(q,m)
    return engines
def scry_quantiles(engines, X, q_list):
    raw = np.vstack([engines[q].model.predict(X) for q in q_list]).T
    out = np.zeros_like(raw)
    for i in range(raw.shape[0]):
        ir = IsotonicRegression(increasing=True, out_of_bounds="clip")
        out[i,:] = ir.fit_transform(q_list, raw[i,:])
    return out
def invert_for_cdf(q_list, q_values, t_grid):
    n, m = q_values.shape
    F = np.zeros((n, len(t_grid)))
    for j,t in enumerate(t_grid):
        for i in range(n):
            if t <= q_values[i,0]: F[i,j]=0.0
            elif t >= q_values[i,-1]: F[i,j]=1.0
            else:
                k = np.searchsorted(q_values[i,:], t)
                t0,t1 = q_values[i,k-1], q_values[i,k]
                q0,q1 = q_list[k-1], q_list[k]
                F[i,j] = q0 if t1==t0 else q0 + (q1-q0)*(t-t0)/(t1-t0)
    return F

# ===== Q2风格分组 =====
@dataclass
class NimbusBin:
    left: float; right: float; n: int
    p50: float; p80: float; p90: float
def rocky_quantile(x: pd.Series, q: float)->float:
    x = pd.to_numeric(x, errors="coerce").dropna()
    return float(np.quantile(x,q)) if len(x)>0 else np.nan
def shard_equal_bins(series: pd.Series, n_bins: int):
    n=n_bins
    while n>=2:

```

```

        try: return pd.qcut(series, q=n, duplicates="drop")
        except Exception: n-=1
    return pd.cut(series, bins=1, include_lowest=True)
def tally_bins(df, bin_col, t_col):
    stats=[]
    for cat, sub in df.groupby(bin_col, observed=True):
        if hasattr(cat,"left"): L,R=float(cat.left), float(cat.right)
        else: L,R=float(sub["BMI"].min()), float(sub["BMI"].max())
        t = pd.to_numeric(sub[t_col], errors="coerce").dropna()
        if len(t)==0:
            stats.append(NimbusBin(L,R,len(sub),np.nan,np.nan,np.nan))
        else:
            stats.append(NimbusBin(L,R,len(sub),
                                   rocky_quantile(t,0.5),
                                   rocky_quantile(t,0.8),
                                   rocky_quantile(t,0.9)))

    return sorted(stats, key=lambda z:z.left)
def fuse_bins_p80(stats: List[NimbusBin], tol_days:int):
    if len(stats)<=1: return stats
    changed=True; merged=stats
    while changed and len(merged)>1:
        changed=False; new=[]
        i=0
        while i<len(merged):
            if i<len(merged)-1:
                a,b = merged[i], merged[i+1]
                if not (np.isnan(a.p80) or np.isnan(b.p80)) and
abs(a.p80-b.p80)<=tol_days:
                    L,R=a.left,b.right; n=a.n+b.n
                    p50=np.nanmean([a.p50,b.p50]); p80=np.nanmean([a.p80,b.p80]);
p90=np.nanmean([a.p90,b.p90])
                    new.append(NimbusBin(L,R,n,p50,p80,p90)); i+=2;
            changed=True; continue
            new.append(merged[i]); i+=1
        merged=new
    return merged

# ===== 风险函数 =====
def piecemeal_time_cost(t):
    if t<=LARK_EARLY_FENCE: return TUSK_RISK_WEIGHTS[0]
    if t<=LARK_LATE_FENCE: return TUSK_RISK_WEIGHTS[1]
    return TUSK_RISK_WEIGHTS[2]

def hazy_expected_loss(F_grp: np.ndarray, t_grid: np.ndarray,
                       alpha=FROST_ALPHA, lambda_time=OPAL_WEIGHT_TIME,
                       lambda_det=OPAL_WEIGHT_DET,
lambda_fail=OPAL_WEIGHT_FAIL):
    """
    返回:
    best_idx —— 在  $\Pi(t) \geq \alpha$  的可行域内, 使  $L(t)$  最小的索引

```

```

    L_plot      —— 未加“不可行罚”的曲线（用于展示U型）
    Pi          —— 组平均达标比例曲线
    """
    Pi = F_grp.mean(axis=0)                # 组内平均命中率  $\Pi(t)$ 
    E_tests = 1.0 + (1.0 - Pi)             # 期望检测次数 ~ 首检命中1次，
    否则2次
    C_time = np.array([piecemeal_time_cost(t) for t in t_grid])

    # —— 关键改动：时间成本不乘 (1-Pi) —— 这样 t 越晚成本越高，确保右侧抬
    升形成U型
    L0 = lambda_time*C_time + lambda_det*E_tests + lambda_fail*(1.0 - Pi)

    # 在可行域内选最小 ( $\Pi \geq \alpha$ )，用于确定 t*
    feasible = (Pi >= alpha)
    L_num = np.where(feasible, L0, np.inf)
    best_idx = int(np.argmin(L_num))
    return best_idx, L0, Pi

# ===== 相邻推荐合并 =====
def fuse_adjacent_recs(df, delta=DELTA_WEEKS_GLUE):
    rows = df.sort_values("t_rec").to_dict("records")
    out=[]
    for r in rows:
        if not out:
            r["groups"]=[r["group"]]; out.append(r); continue
        if abs(r["t_rec"]-out[-1]["t_rec"]) < delta:
            tot=out[-1]["size"]+r["size"]
            out[-1]["t_rec"] = (out[-1]["t_rec"]*out[-1]["size"] + r["t_rec"]*r["size"])/tot
            out[-1]["size"] = tot
            out[-1]["groups"].append(r["group"])
        else:
            r["groups"]=[r["group"]]; out.append(r)
    for r in out:
        r["group"] = "+".join(map(str, r["groups"]))
        del r["groups"]
    return pd.DataFrame(out)

# ===== 主流程 =====
def run_orbital_main():
    mount_hanzi()
    os.makedirs("outputs", exist_ok=True)

    frame_seed = pd.read_excel(QUASAR_LEDGER,
    sheet_name=AURORA_TABCODE)
    harvest = harvest_columns(frame_seed)

    frame = frame_seed.copy()
    frame["GA"] = frame[harvest["ga"]].apply(vapor_parse_ga)
    frame["Y_frac"] = frame[harvest["y_pct"]].apply(morph_fraction)

```



```

frame = frame[~frame["GA"].isna() & ~frame["Y_frac"].isna()].copy()

token_id = harvest["id"]
obelisk_T = (frame.groupby(token_id).apply(lambda g: mint_attain_earliest(g,
PUMPKIN_TAU))
            .rename("T_attain").reset_index())
obelisk_T = obelisk_T.dropna().reset_index(drop=True)

base_pedestal = (frame.sort_values(["GA"]).groupby(token_id)
                .agg({harvest["bmi"]: "first", harvest["age"]: "first" if "age" in harvest else
"first"}))
                .reset_index())
base_pedestal.columns = [token_id, "BMI", "Age"]

pcs, var = brew_quality_pca(frame, harvest)
df_pcs = pd.concat([frame[[token_id]].reset_index(drop=True),
pcs.reset_index(drop=True)], axis=1)
pcs_agg =
df_pcs.groupby(token_id)[["PC1", "PC2", "PC3", "PC4"]].mean().reset_index()

data = (obelisk_T.merge(base_pedestal, on=token_id, how="inner")
        .merge(pcs_agg, on=token_id, how="inner"))
data =
data.dropna(subset=["T_attain", "BMI", "PC1", "PC2", "PC3", "PC4"]).reset_index(drop=True
)

data["T_days"] = data["T_attain"]*7.0
data["BMI_val"] = data["BMI"].astype(float)

for c in ["BMI", "Age"]:
    mu, sd = data[c].mean(), data[c].std(ddof=0)
    data[c+"_z"] = (data[c]-mu)/(sd if sd>0 else 1.0)

# ===== 构造特征： 可选包含 Age_z =====
rivet_feats = ["BMI_z", "PC1", "PC2", "PC3", "PC4"]
if "Age_z" in data.columns and data["Age_z"].notna().any():
    rivet_feats.append("Age_z")

Xmat = data[rivet_feats].values
ytarget = data["T_attain"].values
print(f'[INFO] 训练样本量: {len(data)}; 特征列: {rivet_feats}')

# 分位数回归 → 分位数矩阵 → 反演CDF
q_engines = forge_quantile_engines(Xmat, ytarget, GRAIL_QUANTILES)
Qhat = scry_quantiles(q_engines, Xmat, GRAIL_QUANTILES)
Fhat = invert_for_cdf(GRAIL_QUANTILES, Qhat, TUNDRA_GRID)

# ===== 其它指标对“最优时点”的解释性分析
=====

```

```

# 与组一致的时间代价分段
def _piecemeal_time_cost(t):
    if t <= LARK_EARLY_FENCE: return TUSK_RISK_WEIGHTS[0]
    if t <= LARK_LATE_FENCE: return TUSK_RISK_WEIGHTS[1]
    return TUSK_RISK_WEIGHTS[2]

def _best_t_for_curve(Pi_curve: np.ndarray, t_grid: np.ndarray,
alpha=FROST_ALPHA):
    C_time = np.array([_piecemeal_time_cost(t) for t in t_grid])
    E_tests = 1.0 + (1.0 - Pi_curve) # 首检+可能复检 (简化一致)
    L0 = OPAL_WEIGHT_TIME * C_time + OPAL_WEIGHT_DET * E_tests +
OPAL_WEIGHT_FAIL * (1.0 - Pi_curve)
    feasible = (Pi_curve >= alpha) # 仅在  $\Pi \geq \alpha$  的可行域内选
    L_num = np.where(feasible, L0, np.inf)
    j = int(np.argmin(L_num))
    return j, float(t_grid[j])

# (1) 个体级 t*_i 与 t_rec,i
indi_tstar_list, indi_trec_list = [], []
for i in range(Fhat.shape[0]):
    Pi_i = Fhat[i, :]
    _, t_star_i = _best_t_for_curve(Pi_i, TUNDRA_GRID, alpha=FROST_ALPHA)
    indi_tstar_list.append(t_star_i)
    idx = np.where(Pi_i >= FROST_ALPHA)[0]
    t_rec_i = float(TUNDRA_GRID[idx[0]]) if len(idx) > 0 else np.nan
    indi_trec_list.append(t_rec_i)

data["t_star_individual"] = indi_tstar_list
data["t_rec_individual"] = indi_trec_list
data[[token_id, "BMI", "Age", "PC1", "PC2", "PC3", "PC4", "t_star_individual",
"t_rec_individual"]].to_csv(
    "outputs/t_star_individual.csv", index=False, encoding="utf-8-sig"
)

# —— 置换重要性 —— #
feat_names = rivet_feats.copy() # 与模型输入完全一致

def _recompute_tstar_for_X(X_new: np.ndarray):
    Qp = scry_quantiles(q_engines, X_new, GRAIL_QUANTILES)
    Fp = invert_for_cdf(GRAIL_QUANTILES, Qp, TUNDRA_GRID)
    out = []
    for i in range(Fp.shape[0]):
        _, tstar = _best_t_for_curve(Fp[i, :], TUNDRA_GRID,
alpha=FROST_ALPHA)
        out.append(tstar)
    return np.array(out)

base_tstar = np.array(indi_tstar_list)
imp_rows = []

```

```

R = 8 # 重复次数

for name in feat_names:
    try:
        col_idx = rivet_feats.index(name)
    except ValueError:
        continue
    diffs = []
    for _ in range(R):
        X_perm = Xmat.copy()
        X_perm[:, col_idx] = np.random.permutation(X_perm[:, col_idx])
        tstar_perm = _recompute_tstar_for_X(X_perm)
        diffs.append(np.abs(tstar_perm - base_tstar))
    imp = float(np.mean(diffs))
    imp_rows.append([name, imp])

imp_df = (pd.DataFrame(imp_rows, columns=["feature",
"permutation_importance(| $\Delta t^*$ |, weeks)"]))
        .sort_values("permutation_importance(| $\Delta t^*$ |, weeks)", ascending=False))
imp_df.to_csv("outputs/feature_importance_permutation.csv", index=False,
encoding="utf-8-sig")

# —— 一维偏依赖 —— #
def _pdp_1d(col_name: str, q_grid=(0.1, 0.25, 0.5, 0.75, 0.9)):
    if col_name not in data.columns:
        return None
    grid_vals = np.quantile(data[col_name].dropna().values, q_grid)
    rows = []
    for v in grid_vals:
        X_pd = Xmat.copy()
        X_pd[:, :] = np.median(Xmat, axis=0) # 其它特征固定中位数
        j = rivet_feats.index(col_name) # col_name 一定来自 rivet_feats
        X_pd[:, j] = v
        Qp = sctry_quantiles(q_engines, X_pd, GRAIL_QUANTILES)
        Fp = invert_for_cdf(GRAIL_QUANTILES, Qp, TUNDRA_GRID)
        Pi_bar = Fp.mean(axis=0)
        _, t_pd = _best_t_for_curve(Pi_bar, TUNDRA_GRID,
alpha=FROST_ALPHA)
        rows.append([float(v), float(t_pd)])
    return pd.DataFrame(rows, columns=[col_name, "t_star_pd"])

for name in rivet_feats: # 直接遍历模型实际用到的特征
    pdf = _pdp_1d(name)
    if pdf is None:
        continue
    pdf.to_csv(f"outputs/pdp_{name}.csv", index=False, encoding="utf-8-sig")
    plt.figure(figsize=(6.0, 4.0))
    plt.plot(pdf[name], pdf["t_star_pd"], marker="o")
    plt.xlabel(name.replace("_z", " (z 分) "))

```

```

plt.ylabel("风险最小孕周 t* (周)")
plt.title(f'偏依赖: {name} 对 t* 的影响')
plt.tight_layout()
plt.savefig(f'outputs/plot_pdp_{name}.png', dpi=160)
plt.close()

# —— 分箱对比: 对每个“实际用于建模”的特征分箱 —— #
rows = []
for name in rivet_feats: # 与模型输入一致
    s = pd.qcut(data[name], q=4, labels=["Q1", "Q2", "Q3", "Q4"])
    for ql in ["Q1", "Q2", "Q3", "Q4"]:
        sub = data.loc[s == ql]
        rows.append([
            name, ql, int(len(sub)),
            float(np.nanmean(sub["t_star_individual"])),
            float(np.nanmean(sub["t_rec_individual"]))
        ])
    bins_tab = (pd.DataFrame(rows, columns=["feature", "bin", "n", "mean_t_star",
"mean_t_rec"]))
        .sort_values(["feature", "bin"]))
    bins_tab.to_csv("outputs/tstar_by_feature_bins.csv", index=False,
encoding="utf-8-sig")

# —— 综合一图: 强度 (置换重要性) + 方向性 (Q4-Q1 的 t* 差) —— #
dir_rows = []
for name in rivet_feats:
    s = pd.qcut(data[name], q=4, labels=["Q1", "Q2", "Q3", "Q4"])
    g = data.groupby(s)["t_star_individual"].mean()
    if set(g.index) == set(["Q1", "Q2", "Q3", "Q4"]):
        dir_rows.append([name, float(g["Q4"] - g["Q1"])])
    dir_df = pd.DataFrame(dir_rows, columns=["feature",
"direction(Q4-Q1)"]).set_index("feature")

sum_df = (imp_df.set_index("feature")
        .join(dir_df, how="left")
        .reset_index())

plt.figure(figsize=(7.2, 4.6))
ax = plt.gca()
ax.bar(sum_df["feature"], sum_df["permutation_importance(|Δt*|, weeks)"])
ax.set_ylabel("置换重要性 (|Δt*|, 周)")
ax.set_xlabel("特征 (z 分或主成分)")
ax2 = ax.twinx()
ax2.plot(sum_df["feature"], sum_df["direction(Q4-Q1)"], marker="o")
ax2.set_ylabel("方向性: Q4 - Q1 的平均 t* (周)")
plt.title("影响强度 + 方向性: 特征对最优检测孕周 t* 的作用")
plt.tight_layout()
plt.savefig("outputs/plot_summary_importance_and_direction.png", dpi=160)
plt.close()

```

—— Q2风格: BMI 等频细箱→按p80≤2天相邻合并 (用已达标者决定边界)

```
reached = data[["BMI_val", "T_days"]].dropna().copy()
init_bins = shard_equal_bins(reached["BMI_val"], n_bins=WOBBLE_BINS_INIT)
reached["BMI_bin_init"] = init_bins
init_stats = tally_bins(reached.rename(columns={"BMI_val": "BMI"}),
"BMI_bin_init", "T_days")
merged_stats = fuse_bins_p80(init_stats, tol_days=SAGE_P80_TOL)
```

—— 保证打印边界“连续无缝”的补丁 (对齐到 3 位小数) ——

DEC = 3 # 显示的小数位

```
raw_edges = [merged_stats[0].left] + [s.right for s in merged_stats]
```

```
edges = [round(raw_edges[0], DEC)]
```

```
for i in range(1, len(raw_edges)):
```

```
    e = round(raw_edges[i], DEC)
```

```
    if e < edges[-1]:
```

```
        e = edges[-1]
```

```
    edges.append(e)
```

```
final_bins = [(edges[i], edges[i + 1]) for i in range(len(edges) - 1)]
```

```
def map_to_final_bin_continuous(bmi):
```

```
    for (L, R) in final_bins:
```

```
        if bmi > L and bmi <= R:
```

```
            return f'({L:. {DEC}f}, {R:. {DEC}f})'
```

```
    if bmi <= final_bins[0][0]:
```

```
        L, R = final_bins[0];
```

```
        return f'({L:. {DEC}f}, {R:. {DEC}f})'
```

```
    L, R = final_bins[-1];
```

```
    return f'({L:. {DEC}f}, {R:. {DEC}f})'
```

```
data["BMI_group_label"] = data["BMI_val"].apply(map_to_final_bin_continuous)
```

```
labels_sorted = [f'({L:. {DEC}f}, {R:. {DEC}f})' for (L, R) in final_bins]
```

```
label2code = {lab: i for i, lab in enumerate(labels_sorted)}
```

```
data["group_leaf"] = data["BMI_group_label"].map(label2code)
```

```
hard_rows = []
```

```
for gid, (L, R) in enumerate(final_bins):
```

```
    s = merged_stats[min(gid, len(merged_stats) - 1)]
```

```
    hard_rows.append({
```

```
        "group_leaf": gid,
```

```
        "BMI_edge_left": L,
```

```
        "BMI_edge_right": R,
```

```
        "n_in_group": s.n,
```

```
        "T_p50_days": round(s.p50, 1),
```

```
        "T_p80_days": round(s.p80, 1),
```

```
        "T_p90_days": round(s.p90, 1),
```

```
        "T_p80_weeks": round(s.p80 / 7.0, 2)
```

```
    })
```

```

hard_df = pd.DataFrame(hard_rows).sort_values("group_leaf")
hard_df.to_csv("outputs/bmi_groups_q2_style.csv", index=False,
encoding="utf-8-sig")

final_bins = [(s.left, s.right) for s in merged_stats]
def map_to_bin(b):
    for L,R in final_bins:
        if b>=L and b<=R: return f'({L:.3f}, {R:.3f})'
    return f'({final_bins[0][0]:.3f}, {final_bins[0][1]:.3f})' if b<final_bins[0][0] else \
        f'({final_bins[-1][0]:.3f}, {final_bins[-1][1]:.3f})'
data["BMI_group_label"] = data["BMI_val"].apply(map_to_bin)
labels_sorted = sorted(set(data["BMI_group_label"]), key=lambda s:
float(s.split(",")[0][1:]))
label2code = {lab:i for i,lab in enumerate(labels_sorted)}
data["group_leaf"] = data["BMI_group_label"].map(label2code)
print("[Q2分组] 最终 BMI 区间: ", labels_sorted)

rows=[]
for s in merged_stats:
    lab=f'({s.left:.3f}, {s.right:.3f})'; gid=label2code[lab]

rows.append({"group_leaf":gid,"BMI_edge_left":round(s.left,3),"BMI_edge_right":round(s
.right,3),
            "n_in_group":int(s.n),

"T_p50_days":round(s.p50,1),"T_p80_days":round(s.p80,1),"T_p90_days":round(s.p90,1),
            "T_p80_weeks":round(s.p80/7.0,2)})
pd.DataFrame(rows).sort_values("group_leaf").to_csv(
    "outputs/bmi_groups_q2_style.csv", index=False, encoding="utf-8-sig"
)
print("[输出] Q2风格 BMI 分组 -> outputs/bmi_groups_q2_style.csv")

# —— 逐组求最优点 t*、并收集  $\Pi$  曲线 ——
recs=[]; group_curves={}
for g, idx in data.groupby("group_leaf").indices.items():
    idx = np.array(list(idx))
    F_grp = Fhat[idx,:]
    best_j, L_plot, Pi = hazy_expected_loss(F_grp, TUNDRA_GRID,
alpha=FROST_ALPHA,

lambda_time=OPAL_WEIGHT_TIME,

lambda_det=OPAL_WEIGHT_DET,
lambda_fail=OPAL_WEIGHT_FAIL)

    t_star = float(TUNDRA_GRID[best_j])
    pi_star = float(Pi[best_j])
    n_g = int(F_grp.shape[0])
    recs.append({"group":int(g), "size":n_g, "t_rec_star":t_star,
"Pi(t_rec_star)":round(pi_star,3)})

```

```

        group_curves[int(g)] = {"t":TUNDRA_GRID.copy(), "Pi":Pi.copy(),
"Loss":L_plot.copy()}

        if not QUILL_PLOTS_ONLY:
            plt.figure(figsize=(6,4.2))
            plt.plot(TUNDRA_GRID, Pi); plt.axhline(FROST_ALPHA, linestyle="--");
plt.axvline(t_star, linestyle=":")
            plt.title(f'Group {g} 达标比例  $\Pi_g(t)$ '); plt.xlabel("孕周 t (周)");
plt.ylabel(" $\Pi_g(t)$ ")
            plt.tight_layout(); plt.savefig(f'outputs/group_{g}_Pi_curve.png', dpi=150);
plt.close()

            plt.figure(figsize=(6,4.2))
            plt.plot(TUNDRA_GRID, L_plot); plt.axvline(t_star, linestyle=":")
            plt.title(f'Group {g} 风险函数  $_{g}(t)$ '); plt.xlabel("孕周 t (周)");
plt.ylabel("期望风险")
            plt.tight_layout(); plt.savefig(f'outputs/group_{g}_Loss_curve.png',
dpi=150); plt.close()

        rec_df =
pd.DataFrame(recs).rename(columns={"t_rec_star":"t_star","Pi(t_rec_star)":"Pi(t_star)"}
        rec_df = rec_df.sort_values("t_star").reset_index(drop=True)
        rec_df.to_csv("outputs/recommendations_raw.csv", index=False,
encoding="utf-8-sig")

        merged = fuse_adjacent_recs(rec_df.rename(columns={"t_star":"t_rec"}),
delta=DELTA_WEEKS_GLUE)
        merged.to_csv("outputs/recommendations.csv", index=False, encoding="utf-8-sig")

        bmi_stats = (data.groupby("group_leaf")
            .agg(size=("BMI","size"),
                BMI_min=("BMI","min"), BMI_Q1=("BMI",lambda
s:s.quantile(0.25)),
                BMI_median=("BMI","median"), BMI_Q3=("BMI",lambda
s:s.quantile(0.75)),
                BMI_max=("BMI","max"))
            .reset_index())
        bmi_stats = bmi_stats.merge(rec_df, left_on="group_leaf", right_on="group",
how="left").drop(columns=["group"])
        bmi_stats.to_csv("outputs/bmi_ranges_by_group.csv", index=False,
encoding="utf-8-sig")
        print("[输出] 每组 BMI 经验范围 -> outputs/bmi_ranges_by_group.csv")

        hard = pd.read_csv("outputs/bmi_groups_q2_style.csv")
        q = np.linspace(0,1,len(hard)+1)
        edges = list(np.quantile(data["BMI"].values, q))
        edges = sorted(np.unique(edges))
        nom_rows=[]
        for k in range(len(hard)):
            nom_rows.append({"group_leaf":k, "BMI edge left":edges[k],

```

```

"BMI_edge_right":edges[min(k+1,len(edges)-1)]})
bmi_bins_table = (pd.DataFrame(nom_rows)
                    .merge(bmi_stats, on="group_leaf", how="left")
                    .merge(rec_df[["group", "t_star", "Pi(t_star)"]],
left_on="group_leaf", right_on="group", how="left")
                    .drop(columns=["group"])))
bmi_bins_table.to_csv("outputs/bmi_bins_boundaries.csv", index=False,
encoding="utf-8-sig")
print("[输出] BMI 分箱边界 -> outputs/bmi_bins_boundaries.csv")

data[[token_id, "BMI", "BMI_z", "PC1", "PC2", "PC3", "PC4", "group_leaf"]].to_csv(
    "outputs/membership.csv", index=False, encoding="utf-8-sig"
)

# ===== 表格 =====
def t_at_level(Pi, t_grid, level=0.90):
    idx = np.where(Pi >= level)[0]
    return float(t_grid[idx[0]]) if len(idx)>0 else np.nan

paper_rows=[]
for g in sorted(group_curves.keys()):
    cur = group_curves[g]
    t90 = t_at_level(cur["Pi"], cur["t"], level=FROST_ALPHA)
    star = float(rec_df.loc[rec_df["group"]==g, "t_star"].values[0])
    pi_at_t90 = float(cur["Pi"][np.argmin(abs(cur["t"]-t90))]) if np.isfinite(t90) else
np.nan
    hard_row = hard.loc[hard["group_leaf"]==g].iloc[0]
    bmi_interval = f'({hard_row["BMI_edge_left"]:.1f},
{hard_row["BMI_edge_right"]:.1f})'
    size = int(rec_df.loc[rec_df["group"]==g, "size"])
    paper_rows.append([bmi_interval, size, t90, round(pi_at_t90,3), star])

paper_df = pd.DataFrame(paper_rows, columns=["BMI区间", "样本数", "推荐孕周
(t_rec)", "成功概率 $\Pi(t_{rec})$ ", "风险最小孕周(t*)"])
paper_df = paper_df.sort_values("推荐孕周(t_rec)").reset_index(drop=True)
paper_df.to_csv("outputs/bmi_summary_for_paper.csv", index=False,
encoding="utf-8-sig")
print("[论文用] 汇总表格 -> outputs/bmi_summary_for_paper.csv")

# ===== 图 =====
group2interval = {
    int(row["group_leaf"]): f'({row["BMI_edge_left"]:.1f},
{row["BMI_edge_right"]:.1f})'
    for _, row in hard.iterrows()
}

order = sorted(group_curves.keys(),
key=lambda gid: float(group2interval[gid].split(",")[0][1:]))

```



```

colors = plt.cm.Set2(np.linspace(0, 1, len(order)))

plt.figure(figsize=(7.6, 5.2))
handles, labels = [], []

for i, gid in enumerate(order):
    t = group_curves[gid]["t"]
    Pi = group_curves[gid]["Pi"]

    n_g = int(rec_df.loc[rec_df["group"] == gid, "size"])
    label = f"{group2interval[gid]} (n={n_g})"
    h, = plt.plot(t, Pi, label=label, color=colors[i])
    handles.append(h);
    labels.append(label)

    t_star = float(rec_df.loc[rec_df["group"] == gid, "t_star"])
    pi_star = float(Pi[np.argmin(abs(t - t_star))])
    plt.scatter([t_star], [pi_star], s=28, color=colors[i])

    idx90 = np.where(Pi >= FROST_ALPHA)[0]
    if len(idx90) > 0:
        t90 = float(t[idx90[0]])
        pi90 = float(Pi[idx90[0]])
        plt.scatter([t90], [pi90], marker="^", s=32, color=colors[i])

plt.axhline(FROST_ALPHA, linestyle="--", color="steelblue")

plt.xlabel("孕周 t (周) ")
plt.ylabel("成功概率  $\Pi_g(t)$ ")
plt.title("不同 BMI 区间的首检成功概率曲线")

from matplotlib.lines import Line2D
symbol_handles = [
    Line2D([0], [0], marker='o', color='w', markerfacecolor='black', markersize=6,
label="● 风险最小孕周 t*"),
    Line2D([0], [0], marker='^', color='w', markerfacecolor='black', markersize=7,
label="▲ 首次 ≥90% 孕周 t_rec")
]

plt.legend(handles + symbol_handles, labels + [h.get_label() for h in symbol_handles],
           frameon=False, loc="lower right", fontsize=9)

plt.tight_layout()
plt.savefig("outputs/plot_Pi_all_groups.png", dpi=160)
plt.close()

# ===== 再次声明 =====
rivet_feats = ["BMI_z", "PC1", "PC2", "PC3", "PC4"]
if "Age_z" in data.columns and data["Age_z"].notna().any():

```

```

        rivet_feats.append("Age_z")

Xmat = data[rivet_feats].values
ytarget = data["T_attain"].values
print(f"[INFO] 训练样本量: {len(data)}; 特征列: {rivet_feats}")

print("[论文用] 合并  $\Pi$  曲线图 -> outputs/plot_Pi_all_groups.png")

if __name__ == "__main__":
    run_orbital_main()

```

附录七 q4. py

介绍：规则优先的混合型二分类诊断模型

```

from pathlib import Path
import warnings, numpy as np, pandas as pd
from sklearn.mixture import GaussianMixture
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import RobustScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.calibration import CalibratedClassifierCV
warnings.filterwarnings("ignore")

# ===== 配置 =====
NEBULA_INGEST = r"附件.xlsx"
QUAD_OUTBOX = Path(r"out_fourth")
ORBIT_FOLDS = 5
CHAMELEON_FP, CHAMELEON_FN = 1.0, 10.0
QUAD_OUTBOX.mkdir(parents=True, exist_ok=True)

# 读数据
atlas = pd.read_excel(NEBULA_INGEST)
atlas.columns = [c.strip() for c in atlas.columns]
halo_labels = atlas['染色体的非整倍体'].notna().astype(int)

# 女胎筛选 (GMM)
def minty_selector(series: pd.Series):
    s = series.fillna(series.median()).values.reshape(-1,1)
    gmm = GaussianMixture(n_components=2, random_state=42).fit(s)
    fem_comp = int(np.argmax(gmm.means_.flatten()))
    probs = gmm.predict_proba(s)[:, fem_comp]
    return (probs >= 0.5, probs)

```

```

veil, nebulae = minty_selector(atlas['Y染色体浓度'])
atlas['female_prob'] = nebulae
atlas_phi = atlas[veil].copy()
halo_phi = halo_labels[veil].copy()

# 特征工程
def crank_gadgets(frame: pd.DataFrame) -> pd.DataFrame:
    f = frame.copy()
    for col in ['13号染色体的GC含量','18号染色体的GC含量','21号染色体的GC含量',
                'GC含量']:
        if col in f: f[col + '_偏离'] = (f[col] - 0.5).abs()
        if '原始读段数' in f: f['log_原始读段数'] = np.log1p(f['原始读段数'])
        if '唯一比对的读段数' in f: f['log_唯一比对的读段数'] = np.log1p(f['唯一比对的读段数'])
    return f

atlas_phi = crank_gadgets(atlas_phi)
baseline_roster = ['13号染色体的Z值','18号染色体的Z值','21号染色体的Z值','X染色体的Z值',
                   'GC含量','13号染色体的GC含量','18号染色体的GC含量','21号染色体的GC含量',
                   '被过滤掉读段数的比例','在参考基因组上比对的比例','原始读段数','唯一比对的读段数',
                   '孕妇BMI','年龄']
gizmo_cols = [c for c in atlas_phi.columns if c.endswith('偏离') or c.startswith('log_')]
astro_cols = [c for c in baseline_roster + gizmo_cols if c in atlas_phi.columns]
matrix_X = atlas_phi[astro_cols].copy().fillna(atlas_phi[astro_cols].median())

# 规则层
def oracle_glimpse(row):
    z13 = row.get('13号染色体的Z值', np.nan)
    z18 = row.get('18号染色体的Z值', np.nan)
    z21 = row.get('21号染色体的Z值', np.nan)
    zs = np.array([z13, z18, z21], dtype=float)
    if np.any(zs >= 3.0): return 0.995
    if np.all(zs <= -3.0): return 0.005
    return np.nan
omen_series = matrix_X.apply(oracle_glimpse, axis=1)

# 模型与校准
prism_scaler = RobustScaler()
comet_lr = LogisticRegression(max_iter=1000, class_weight='balanced', solver='liblinear')
geyser_gb = GradientBoostingClassifier()
glaze_lr = CalibratedClassifierCV(comet_lr, method='sigmoid', cv=3)
glaze_gb = CalibratedClassifierCV(geyser_gb, method='isotonic', cv=3)

```

```

def pick_threshold(y_true, p, cfp=CHAMELEON_FP, cfn=CHAMELEON_FN):
    grid = np.linspace(0, 1, 1001)
    best_tau, best_cost = 0.5, float('inf')
    for tau in grid:
        yhat = (p >= tau).astype(int)
        fp = ((y_true == 0) & (yhat == 1)).sum()
        fn = ((y_true == 1) & (yhat == 0)).sum()
        cost = cfp*fp + cfn*fn
        if cost < best_cost:
            best_cost, best_tau = cost, float(tau)
    return best_tau

# 交叉验证与 OOF
phase_splitter = StratifiedKfold(n_splits=ORBIT_FOLDS, shuffle=True,
random_state=42)
ooze_pred = pd.Series(index=matrix_X.index, dtype=float)
ledger_folds = []

for epoch, (forge, assay) in enumerate(phase_splitter.split(matrix_X, halo_phi), 1):
    X_forge, X_assay = matrix_X.iloc[forge], matrix_X.iloc[assay]
    y_forge, y_assay = halo_phi.iloc[forge], halo_phi.iloc[assay]
    duct_lr = Pipeline([('scaler', prism_scaler), ('clf', glaze_lr)])
    duct_gb = Pipeline([('clf', glaze_gb)])
    duct_lr.fit(X_forge, y_forge); duct_gb.fit(X_forge, y_forge)
    zeta_lr = duct_lr.predict_proba(X_assay)[: , 1]
    zeta_gb = duct_gb.predict_proba(X_assay)[: , 1]
    zeta_mix = 0.5*zeta_lr + 0.5*zeta_gb
    augury = omen_series.loc[X_assay.index].to_numpy()
    need_model = np.isnan(augury)
    zeta_final = zeta_mix.copy(); zeta_final[~need_model] = augury[~need_model]
    ooze_pred.loc[X_assay.index] = zeta_final

    from sklearn.metrics import roc_auc_score, average_precision_score, brier_score_loss,
confusion_matrix
    helio_auc = roc_auc_score(y_assay, zeta_final)
    prism_ap = average_precision_score(y_assay, zeta_final)
    theta_star = pick_threshold(y_assay.to_numpy(), zeta_final)
    verdict = (zeta_final >= theta_star).astype(int)
    tn_s, fp_s, fn_s, tp_s = confusion_matrix(y_assay, verdict).ravel()
    sense = tp_s/(tp_s+fn_s) if (tp_s+fn_s)>0 else np.nan
    speci = tn_s/(tn_s+fp_s) if (tn_s+fp_s)>0 else np.nan
    brume = brier_score_loss(y_assay, zeta_final)
    ledger_folds.append({'fold': epoch, 'AUC': helio_auc, 'AP': prism_ap, 'Brier': brume,
'tau': theta_star,
                        'tp': tp_s, 'fp': fp_s, 'tn': tn_s, 'fn': fn_s,
                        'sensitivity': sense, 'specificity': speci})

chronicle_cv = pd.DataFrame(ledger_folds)
theta_global = pick_threshold(halo_phi.to_numpy(), ooze_pred.to_numpy())

```

```

from sklearn.metrics import roc_auc_score, average_precision_score, brier_score_loss,
confusion_matrix
verdict_all = (ooze_pred >= theta_global).astype(int)
tn_g, fp_g, fn_g, tp_g = confusion_matrix(halo_phi, verdict_all).ravel()
sense_g = tp_g/(tp_g+fn_g) if (tp_g+fn_g)>0 else np.nan
speci_g = tn_g/(tn_g+fp_g) if (tn_g+fp_g)>0 else np.nan
aurora_auc = roc_auc_score(halo_phi, ooze_pred)
aurora_ap = average_precision_score(halo_phi, ooze_pred)
aurora_brier = brier_score_loss(halo_phi, ooze_pred)

codex = {'N_female': int(veil.sum()), 'Positives': int(halo_phi.sum()),
        'AUC': float(aurora_auc), 'AP': float(aurora_ap), 'Brier': float(aurora_brier),
        'tau_star': float(theta_global), 'sensitivity': float(sense_g), 'specificity':
float(speci_g),
        'tp': int(tp_g), 'fp': int(fp_g), 'tn': int(tn_g), 'fn': int(fn_g)}

chronicle_cv.to_csv(QUAD_OUTBOX/'cv_metrics.csv', index=False)
pd.DataFrame([codex]).to_csv(QUAD_OUTBOX/'summary.csv', index=False)

# 特征重要性（用于后续可视化/解释）
from sklearn.ensemble import GradientBoostingClassifier
duct_lr_full = Pipeline([('scaler', RobustScaler()), ('clf', LogisticRegression(max_iter=1000,
class_weight='balanced', solver='liblinear'))])
duct_gb_full = Pipeline([('clf', GradientBoostingClassifier())])
duct_lr_full.fit(matrix_X, halo_phi); duct_gb_full.fit(matrix_X, halo_phi)
coef_lr_abs = np.abs(duct_lr_full.named_steps['clf'].coef_[0])
imp_gb = duct_gb_full.named_steps['clf'].feature_importances_
pd.DataFrame({'feature': matrix_X.columns, 'LR_abscoef': coef_lr_abs, 'GB_importance':
imp_gb}) \
    .sort_values('GB_importance', ascending=False) \
    .to_csv(QUAD_OUTBOX/'feature_importances.csv', index=False)

# 每样本输出
parchment = atlas_phi[['序号', '孕妇代码']].copy()
parchment['p'] = ooze_pred.values
parchment['decision_tau'] = theta_global
parchment['yhat'] = (ooze_pred >= theta_global).astype(int).values
parchment['rule_applied'] = (~omen_series.isna()).astype(int).values
parchment['female_prob'] = atlas_phi['female_prob'].values
parchment['label_y'] = halo_phi.values
parchment.to_csv(QUAD_OUTBOX/'predictions.csv', index=False)

print("结果文件已保存到：", QUAD_OUTBOX.resolve())
print("Summary:", codex)

```

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	logitV	孕周周	BMI	年龄	季节	ID	唯一比对该段数_log	原始该段数_log	比比例	重复比例	过滤比例	GC	PC1	PC2	PC3	PC4	
1	-3.628581	11.857142	28.125	31春	A001	15.162391047276952	15.433022386815946	0.8067259	0.0276035	0.0274838	0.3992619	1.1773605	0.8015335	-1.49824	0.10677580134015806		
2	-3.320135	15.857142	28.51625	31春	A001	14.714615252283593	14.9782894236073	0.8063927	0.0282708	0.0196167	0.3932988	-2.984826	-0.110325	-2.037985	-1.356113		
3	-2.647051	20.142857	28.51625	31夏	A001	14.889246650524923	15.163283921073136	0.8038578	0.0325962	0.0233124	0.3998897	-1.299625	-0.361445	0.3011888	-0.444971		
4	-2.736001	22.857142	28.90025	31夏	A001	15.321710367494969	15.600626172251202	0.8025347	0.0347616	0.0232802	0.3979775	1.7061665	-0.754796	0.8777401	-1.891802		
5	-2.765208	13.857142	33.331832	32冬	A002	14.969692611519244	15.239654981786678	0.8050077	0.0285551	0.0242115	0.40306	-0.369464	0.0557478	-0.45441	1.1376733173495104		
6	-3.117247	16.714285	34.232692	32春	A002	15.153884333910083	15.44644378194298	0.7894339	0.0302902	0.0260175	0.3994663	0.9103692	1.0966327	1.232072	-0.441549		
7	-2.991204	19.714285	33.782262	32春	A002	15.210973508813783	15.486281358981735	0.7986666	0.0294676	0.0206205	0.3968289	0.6863740	-0.406883	-0.708191	1.191512		
8	-3.110537	23.571428	34.232692	32春	A002	15.339914627707342	15.615296892208846	0.809561	0.0311938	0.0275782	0.3988559	2.3558157	0.3025088	-0.370662	-0.901763		
9	-3.00141	23.571428	34.232692	32春	A002	14.579326338526887	14.869274005236187	0.7916365	0.0285592	0.02695	0.4071893	-2.663247	1.6612674	0.3523683	2.7952014111868375		
10	-2.849398	13.30	34.212188	35春	A003	15.11154180638238	15.402805105712684	0.7876009	0.0306553	0.0211379	0.403697	0.2948671	-0.083431	0.0920260	0.697896687079708		
11	-2.811395	13.30	34.212188	35春	A003	14.88153525891405	15.153536256409186	0.802652	0.0305616	0.0209069	0.4035087	-1.320947	-0.605258	0.313936	0.9857041907018133		
12	-2.663122	13.30	34.212188	35春	A003	15.08261195244206	15.353787321324226	0.8030206	0.0305499	0.0205598	0.404156	0.0746097	-0.978563	0.4857978	0.9513876898680317		
13	-2.695331	16.714285	31.140625	35夏	A003	15.016479053871697	15.294343534376711	0.7982068	0.0296688	0.022111	0.4011541	-0.345131	-0.065775	-0.098913	0.31608652472578463		
14	-2.897991	20.285714	31.852813	35夏	A003	15.134303880809523	15.405600909191557	0.8067726	0.0282744	0.0262064	0.3986279	0.8252421	0.5181135	-1.352796	-0.191846		
15	-2.994878	12.142857	35.883635	31秋	A006	14.7082390681155	14.980747315667129	0.8004625	0.0284419	0.0208677	0.3988536	-2.692374	0.1677857	-0.980298	0.22031803548688791		
16	-2.855964	11.28	64.1243	26秋	A004	14.924661123423341	15.189589275685322	0.8065957	0.0283491	0.021022	0.4052816	-0.899534	-0.703772	-0.249489	1.900224502111627		
17	-2.636917	15.857142	28.61243	26秋	A004	15.2664904606637	15.547954995405085	0.798637	0.0306337	0.0251807	0.4038984	1.8035261	0.0368661	0.5608369	0.6905457577603581		
18	-2.909411	19.285714	28.440955	26秋	A004	15.223437488661242	15.536940369747823	0.7705792	0.0282602	0.0239318	0.3994962	1.2029697	1.7251012	0.4197373	-0.236054		
19	-2.410878	23.571428	29.161993	26秋	A004	15.218833850298246	15.53946502134219	0.7663174	0.0274518	0.0262848	0.401436	1.4911422	0.4435777	0.5461795	0.678323821716587		
20	-2.911865	12.285714	29.955556	30夏	A005	14.7921649834498425	15.067482932030653	0.800899	0.0277375	0.0252101	0.3974382	-1.717457	1.1375252	-1.545822	-0.095005		
21	-2.862899	15.85714	30.844444	30夏	A005	15.089169953098262	15.368559376462007	0.7974001	0.0265857	0.0257036	0.4001361	0.5257634	1.0374059	-1.248166	0.5719330896670102		
22	-2.177768	14.28571	31.288889	30秋	A005	15.114232748515569	15.378939450161337	0.8100025	0.0297608	0.0234922	0.3995713	0.4265130	-0.41494	-0.80986	-0.196937		
23	-1.624701	23.714285	32.222222	30秋	A005	15.135311955935142	15.405263180217926	0.8067726	0.0282744	0.0262064	0.3986279	0.8252421	0.5181135	-1.352796	-0.191846		
24	-2.994878	12.142857	35.883635	31秋	A006	15.010549978171383	15.280258593685515	0.8090194	0.029771	0.027177	0.4012756	1.3818585	0.4780893	-0.70117	0.4507406606588659		
25	-2.238124	16.28571	36.139946	31秋	A006	15.04538383045956	15.331277805854096	0.7930708	0.0317416	0.0215592	0.3987279	-0.332766	-0.041483	0.4241751	-0.852588		
26	-2.784231	17.428571	33.333333	24冬	A009	15.072391037862932	15.3477051574273	0.8032163	0.0319123	0.0234705	0.4043725	0.283578	-0.488721	0.8053598	0.7988844730189085		
27	-2.799251	20.857142	34.222222	23冬	A009	15.17723119317649	15.472252006273827	0.7879914	0.0293227	0.0266304	0.3991973	1.1434471	1.3696882	-0.159762	-0.360158		
28	-1.663689	13.857142	34.666667	24冬	A009	15.035714324660666	15.375262304396948	0.7559714	0.0377558	0.0210838	0.4056452	-0.302803	0.7991733	0.822136	-0.172453		
29	-3.69681	13.857142	33.333333	33春	A010	15.007737954249603	15.26752401127708	0.8090452	0.0274764	0.0198256	0.3989857	-0.674864	-0.775776	-1.449317	0.14223281228078394		
30	-4.594727	16.571428	33.333333	23夏	A010	15.273482583287604	15.538505342954172	0.811301	0.030629	0.0244937	0.4023679	1.7392660	-0.691086	-0.168069	0.29140981230599267		

附录十一 模型指标展示.xlsx（部分）

		B	C	D	E	F	G	H	I	J
1	ID	孕周周_z	BMI_z	年龄_z	唯一比对该段数_log_z	原始该段数_log_z	比比例_z	重复比例_z	过滤比例_z	GC_z
2	A001	-1.22278	-1.398416	0.5753137	0.5363503816068642	0.5002407004881969	0.669373843707568	-1.128066435	1.469499352641038	-0.457778992
3	A001	-0.232712	-1.26689	0.5753137	-1.806001418	-1.887600137	0.6425117199868392	-0.850731199	-1.040530341	-2.458337093
4	A001	0.8280747	-1.26689	0.5753137	-0.892490294	-0.916193625	0.4381515835166464	0.946850629772374	-0.180448997	-0.247158072
5	A001	1.4999063	-1.135364	0.5753137	1.3697627737338793	1.3802914950738887	0.3314850891619923	1.8467636333335847	0.12831626072353272	-0.886801261
6	A002	-0.727746	0.3547587	0.85038234	-0.475489583	-0.515169017	0.530854933186636	-0.607935593	0.4254729936912698	0.816442835533103
7	A002	-0.020555	0.6580843	0.85038234	0.4866199865904954	0.5706794255257653	-0.724679763	-0.011515017	1.00167126709397	-0.389204132
8	A002	0.7219960	0.5064215	0.85038234	0.7904892106542314	0.7798666298988011	0.03576793403967357	-0.353333874	-0.720256621	-1.341120737
9	A002	1.6767040	0.6580843	0.85038234	1.4649907475272355	1.4573274777365341	0.673136310598295	0.3640115127990101	1.4996332725454984	-0.593986643
10	A002	1.6767040	0.6580843	0.85038234	-2.513708789	-2.460040337	-0.547112741	0.730994324	1.2991818299640747	2.2017773332425847
11	A003	-0.939903	-0.517192	1.6755880	0.27035384437713184	0.34153264096416514	-0.87457238	0.14022496383771163	-0.555170931	0.00149354717954
12	A003	-0.939903	-0.517192	1.6755880	-0.932829214	-0.967378633	0.3409416531313071	0.10130502194686516	0.628868349	0.9669767683156433
13	A003	-0.939903	-0.517192	1.6755880	0.11901941272568844	0.08414014767054905	0.3706576759709427	0.0964301589636331	-0.73961641	1.841388246787687
14	A003	-0.020555	-0.383035	1.6755880	-0.226927188	-0.22799308	-0.017424231	-0.269762234	-0.244716396	0.17703427192664561
15	A003	0.8634342	-0.133136	1.6755880	0.38942411640618874	0.3562091196434795	0.4965436490151593	0.21530117849421207	-0.187489121	0.9983490639670367
16	A003	1.5706254	-0.056851	1.6755880	-1.839355759	-1.87469374	0.16442718756593935	-0.779640486	-0.641393095	-0.594758267
17	A004	-1.434937	-1.224593	-0.800029	-0.70723429	-0.778014528	0.658877299557486	-0.818194709	-0.592143113	0.517652894436822
18	A004	-0.232712	-1.224593	-0.800029	1.0809029255855054	1.1037150299751655	0.017257899194367893	0.13124407815419584	0.7346768497497294	1.0977168476678851
19	A004	0.61591734	-1.292032	-0.800029	0.8556893084570505	1.0458772065000526	-2.24472318	-0.855165537	0.33622237728442156	-0.379173009
20	A004	1.6767040	-1.049253	-0.800029	0.8316072997255191	1.059134157556606	-2.588303609	-1.19109887	1.0899662641232187	0.2716086326384698
21	A005	-1.116701	-0.782056	0.3002452	-1.400332433	-1.417326316	0.19961721461288687	-1.072364995	0.7440650852852548	-1.069609456
22	A005	-0.37415	-0.482761	0.3002452	0.15332485042763747	0.16167988617442294	-0.08245927	-1.551036643	0.90133209122547	-0.164493782
23	A005	0.8280747	-0.3002452	0.3002452	0.2844308272306	0.2161956258142727	0.9325088096228271	-0.231523859	0.19598389729198396	-0.353977752
24	A005	1.7120636	-0.018854	0.3002452	0.3946974404512001	0.35443999558043476	0.6731387331991846	-0.849255857	0.0619547405010106	-0.670487144
25	A006	-1.152061	1.2139675	0.5753137	-0.257942667	-0.301959352	0.85422726456140166	-0.227280691	1.371614842382181	0.2177961904313858
26	A006	-0.091274	1.3002693	0.5753137	-0.075723937	-0.034057367	-0.431481456	0.5916755103044193	-0.420771758	-0.63692924
27	A006	0.8634342	1.3290366	0.5753137	0.6608799213827085	0.6198579767517215	0.925821524933475	1.0066165330646606	0.7276018489674844	-0.694532708
28	A007	-1.116701	0.5733171	1.6755880	-2.4373524457043723	-2.4518322478151955	-0.019189778	-0.047962202	-0.103726536	2.679916714830111
29	A007	-0.197352	0.5733171	1.6755880	0.6294728091879408	0.6182954190712791	0.1381415872346423	-0.908502273	-0.102147543	3.388049039148297
30	A007	1.00487246	0.6640609	1.6755880	1.1751662280713648	1.2508148915806166	0.588318226406713	-1.346574999	0.208363677848986	0.1970629676857403
31	A008	-0.939903	-1.057899	-1.07598	-0.661883938	-0.709817274	0.195981959867947	-0.234154539	0.496718177298062	0.550490045312361
32	A008	-0.197352	-0.91778	-1.07598	2.354407157931238	2.319287250869934	0.482467631963892	-0.136362318	-0.017014306	1.5133206718956456
33	A008	0.8280747	-0.56738	-1.07598	1.75372262396445	1.7124383719778272	0.1783093452367662	-0.580207473	0.703444032087512	-0.23833147
34	A008	1.5562639	-0.357157	-1.07598	1.45473582611193	1.4072223505734217	0.7808572981545891	-0.60768624	-0.151239703	-0.105145561
35	A009	-0.3526414	1.350166	-0.465871683	-0.465871683	-0.465871683	0.7327272727272086	0.382564568631594	0.879442828	0.7505993492119
36	A009	1.5624313	0.3526423	-1.05916	0.0655392747458379	0.0520206112346236	0.396437326434599	0.6626290678695922	1.80594162028784	1.2567722020668557
37	A009	1.2523984	0.6454909	-1.625235	0.6139803893305507	0.7061946860769007	-0.409756668	-0.413596814	1.19722841139383	-0.47905864
38	A009	1.7474232	0.4024064	-1.63186	-0.126305913	0.1969053331829932	-0.422838373	0.0911253994468	-0.572446694	1.6837491484208217
39	A010	-0.727746	0.3562645	-1.625235	-0.27265295	-0.368828365	0.8356260959755513	-1.180879528	-0.973882079	-0.550440165
40	A010	-0.055914	0.3526423	-1.625235	1.1174792900800892	1.054094898488326	1.038212085983566	0.1293157436066912	0.5154995470497491	0.5842508701246247